

Multi-Value Plaintext-Checking and Full-Decryption Oracle-Based Attacks on HQC from Offline Templates

Haiyue Dong¹ and Qian Guo²

¹ Independent Researcher, Lund, Sweden, chelseadong202@gmail.com

² Lund University, Lund, Sweden, qian.guo@eit.lth.se

Abstract. The Hamming Quasi-Cyclic (HQC) key encapsulation mechanism (KEM), recently selected by NIST for standardization in the Post-Quantum Cryptography (PQC) process, distinguishes itself through its efficiency, robust design based on hard decoding problems in coding theory, and well-characterized decryption failure rates. Despite its selection, practical security concerns arise from implementation threats, particularly those exploiting plaintext-checking (PC) oracles. While multi-value PC (MV-PC) and full decryption (FD) oracle attacks have been extensively studied in the context of lattice-based cryptography, their applicability to code-based schemes like HQC has remained relatively unexplored.

In this work, we present the first MV-PC and FD oracle attacks targeting code-based KEMs, specifically on HQC. Our MV-PC attack significantly reduces the required oracle queries compared to previous PC oracle-based methods and holds implications for side-channel, key-mismatch, and fault-injection attacks. Our FD attack exhibits remarkable efficiency in trace complexity, achieving secret key recovery for `hqc-128` with just two queries to a perfect oracle, and four queries for `hqc-192` and `hqc-256`. Simulations further demonstrate the robustness of our MV-PC and FD oracle attacks against imperfect oracle responses. We experimentally validate the new attacks on an ARM Cortex-M4 microcontroller, highlighting the critical need for robust countermeasures. In particular, on such a platform, substantial leakage during operations like syndrome computation poses significant challenges to the efficiency of masking techniques in mitigating FD oracle attacks.

Keywords: Code-based cryptography; Side-channel attacks; HQC; Plaintext-checking oracle

1 Introduction

Post-Quantum Cryptography (PQC) has drawn significant attention due to the pressing threat quantum computers pose to traditional cryptographic systems. The National Institute of Standards and Technology (NIST) has been at the forefront of efforts to standardize PQC algorithms, recently introducing the FIPS 203 KEM (Key Encapsulation Mechanism) standard [Nat24] based on CRYSTALS-Kyber [SAB⁺22], along with two new post-quantum digital signature standards. Transitioning the digital infrastructure to post-quantum cryptography has become an urgent priority. For instance, the National Security Memorandum 10 (NSM-10) in the USA sets 2035 as the target year for completing the migration to PQC across Federal systems [Bid22], emphasizing the critical importance of investigating the implementation security of PQC, such as side-channel analysis and fault injection.

Following its evaluation of round-4 candidates—which included the code-based schemes Classic McEliece [ABC⁺22], BIKE [ABB⁺22], Hamming Quasi-Cyclic (HQC) [AAB⁺22],

and the compromised isogeny-based SIKE [JAC⁺22]—NIST selected HQC for standardization in March 2025 [ABC⁺25]. HQC’s security foundation rests on the well-established hardness of decoding problems in coding theory, providing a distinct alternative to lattice-based approaches like Kyber. Furthermore, HQC offers practical benefits such as efficient decapsulation and well-understood decryption failure rates, enhancing resilience against attacks targeting decryption errors.

Earlier HQC proposals [AAB⁺19] employed tensor product codes, some of which were vulnerable to decryption failure attacks [GJ20]. The recent proposal [AAB⁺22], which forms the basis for standardization, uses concatenated codes, combining Reed-Solomon (RS) and duplicated Reed-Muller (RM) codes, which have demonstrated improved resilience.

Given HQC’s selection for standardization by NIST, it is imperative to rigorously evaluate its security claims in practical settings. Specifically, secure implementation strategies must be developed and validated for HQC across diverse real-world platforms, necessitating comprehensive research. A substantial body of work has already investigated HQC’s implementation security [WTBB⁺19, PT19, BDH⁺19, GJN20, XIU⁺21, SHR⁺22, GHJ⁺22, UXT⁺22, HSC⁺23, GLG22, GNNJ23, TUX⁺23, PRJB23, GMGL24, SGG24, DG25]. Among these efforts, attacks leveraging plaintext-checking oracles (PCOs) have emerged as particularly significant.

Plaintext-Checking Oracle (PCO) and Decryption Failures. Plaintext-checking oracles (PCOs), as formally defined in [OP01, ABP15, BDH⁺19], are entities capable of determining whether a given ciphertext c decrypts to a specified message m . This capability allows PCOs to detect decryption or decapsulation errors, which often leak sensitive information about secret keys in post-quantum cryptographic systems. Such vulnerabilities enable attackers to bypass the security guarantees of chosen ciphertext attack (CCA)-secure transformations, like the Fujisaki-Okamoto (FO) transformation [FO99], undermining the security of IND-CCA-secure KEMs.

The application of PCO-type attacks to post-quantum cryptography dates back to reaction attacks proposed by Hall, Goldberg, and Schneier [HGS99]. These attacks leveraged decryption failures to recover messages in the code-based McEliece public-key encryption (PKE) scheme [McE78] and to extract secret keys in early lattice-based cryptosystems, such as Ajtai-Dwork PKE [AD97]. The first reaction-type key recovery attack in code-based cryptography was proposed by Guo, Johansson, and Stankovski [GJS16]. Over time, these attacks have evolved into a major category of mathematical attacks in lattice-based and code-based cryptography. For instance, while CCA transformations such as the FO transform are designed to achieve IND-CCA security, thereby preventing the creation of PCOs in a black-box model, improper parameter choices can compromise the security provided by these transformations, leading to decryption failure attacks [DGJ⁺19]. Moreover, attacks like key mismatch attacks [DFR18] and misuse attacks [BDH⁺19] specifically target ephemeral-key CPA-secure schemes that reuse secret keys.

These attacks have been applied to a broad range of cryptographic primitives, including NTRU [JJ00, HNP⁺03, GN07, GJN19], code-based PKE/KEMs like HQC [GJ20], QC-MDPC [GJS16, WWW23], QC-LDPC [FHS⁺17], and LRPC [AG19], as well as lattice-based schemes like LWE-based PKE/KEMs [Flu16, DFR18, DGJ⁺19, BDH⁺19, GJY19, BGRR19, QCD19, OWT20, GMR20, DRV20, QCZ⁺21, GM23] and advanced FHE schemes [ZPS12, CCCM24].

As research on implementation security has grown, the use of side-channel leakages to detect decryption failures has become increasingly prominent. The idea was discussed and implemented in both code-based cryptography [GJS16, ELPS18, NJW18, PT19] and lattice-based cryptography [DTVV19]. In 2020, Guo, Johansson, and Nilsson [GJN20], along with Ravi et al. [RRCB20], proposed key recovery attacks using side-channel leakage from the FO transformation. By 2021, these attacks expanded to include fault-injection

techniques, where errors were introduced during the ciphertext comparison stage of the FO transform, as demonstrated by Xagawa et al. [XIU⁺21].

Contemporary research on PCO-based attacks focuses on two primary areas. The first involves developing or refining mathematical methods to recover secret keys from PCOs. The second explores the discovery of side-channel leakages or fault-injection techniques to construct PCOs on real-world platforms. These attacks exploit a variety of vulnerabilities, including timing attacks [ELPS18, PT19, DTVV19, GJN20, GHJ⁺22, BBB⁺24], power and electromagnetic (EM) side-channel attacks [RRCB20, UXT⁺22, SHR⁺22], microarchitectural attacks (e.g., DVFS [WPH⁺22], Intel-SGX [HSC⁺23], contention between SMT sibling threads [SGG24], data memory-dependent prefetchers [CWS⁺24], and AMD SEV-SNP [GWSG25]), and Rowhammer-based attacks [MKB⁺24]. Collectively, these studies illustrate the power and generality of the oracle-based attack paradigm. They show that the initial mathematical breakthrough in exploiting an oracle is the critical enabler; once this theoretical foundation is established, the research challenge shifts to discovering physical leakages on diverse platforms to instantiate the required oracle.

Significant progress [GHJ⁺22, SHR⁺22, HSC⁺23, SGG24, GWSG25, DG25] has been made in PCO attacks on the recent HQC proposal [AAB⁺22] with the concatenated RM and RS codes. For example, while early methods required over 800 000 oracle calls [GHJ⁺22] for hqc-128, the OT-PCA attack framework has reduced this number to just 460 [DG25].

Multi-Value PC (MV-PC) Oracle Attacks. A recent development in PCO attacks involves the creation of advanced oracles that exploit strong information leakage, enabling classifiers to extract multiple bits of information per oracle call instead of a simple YES/NO response. This improvement drastically reduces the number of required oracle calls. In the context of lattice-based cryptography, such advancements include parallel PCOs [RRD⁺23], multi-value PC (MV-PC) oracles [TUX⁺23], parallel mismatch attacks [SLZ24, GMÅ24], and Rowhammer-based attacks [MKB⁺24]. These techniques leverage side-channel leakage or fault injection to classify side-channel measurements or deduce the value of $\mathcal{K}(m', aux)$, where m' is the decrypted message, aux represents auxiliary data, and $\mathcal{K}$ is a hash function. By crafting ciphertexts such that m' belongs to a small candidate set, attackers can infer m' through trace comparisons or by comparing the resulting hash function outputs.

While these attacks have achieved significant success in lattice-based KEMs, their application to code-based KEMs remains an open challenge. Factors such as the complex decoding processes, longer code lengths (e.g., $\sim 35\,000$ for hqc-128), and the extremely low Hamming weight of secret vectors in HQC and BIKE make MV-PC oracle attacks in code-based cryptography significantly more difficult. It is noteworthy that Tanaka et al. [TUX⁺23] explicitly stated that their MV-PC attacks have no gain against HQC and BIKE and quantified the required number of traces for hqc-128 to be 36 222.

In this paper, we introduce the first MV-PC oracle attack on HQC, marking a breakthrough in code-based KEMs. Our method employs the offline template (OT) technique—meaning a profile or template is created in advance, without needing online interaction with the target during its generation—drawn from the OT-PCA attack framework, enabling us to exploit a more potent MV-PC oracle. Instead of constructing OT for binary decryption outputs (‘success’ or ‘failure’) as in [DG25], we create OT for multiple decryption outputs provided by the MV-PC oracle. We propose a novel mathematical methodology for recovering HQC’s secret key, achieving a significant reduction in the required number of oracle calls. For instance, our method reduces the number of oracle calls for hqc-128 to 276, 60% of those needed by the best previous PCO approach [DG25]. This novel attack holds significance for side-channel attacks, parallel mismatch attacks, and fault-injection attacks. Simulations demonstrate its resilience even in the presence of oracle response errors.

On the physical side, we evaluated this attack using a ChipWhisperer platform equipped with an ARM Cortex-M4 processor. The evaluation focused on the re-encryption step of

the HQC decapsulation process. By leveraging side-channel leakage and machine learning techniques, we constructed an accurate MV-PC oracle with a single trace. The experimental results aligned with our simulations, validating the effectiveness of our approach.

FD Oracle Attacks. Full decryption (FD) oracles represent a distinct type of PC-related oracle that has been extensively studied in lattice-based cryptography [XPRO20, NDGJ21, REB⁺22, NDJ23, GNNJ23]. By definition, an FD oracle recovers the full message. However, since the entire message space (e.g., 2^{128} or 2^{256} possibilities) is too vast to handle at once, it is infeasible to treat the entire message as a single candidate for trace classification or hash comparison, as did for MV-PC oracles. This requires the secret to be recovered in a parallel manner, piece by piece. FD attacks partition secret information into smaller chunks and recover each chunk independently, enabling the extraction of significantly more information per query. This makes FD oracles the most powerful class of PCO-style attacks, though their construction is challenging, as they require leakage sources that exhibit highly parallel behavior.

Existing research on FD oracles for lattice-based KEMs has primarily focused on side-channel attacks targeting embedded devices, such as the ARM Cortex-M4 platform, where the explored leakage is limited to a small number of clock cycles. FD oracle attacks in lattice-based cryptography have demonstrated remarkable efficiency and resilience against conventional countermeasures like shuffling (randomizing the order of computational steps) and masking (splitting sensitive data into multiple random shares to obscure the original value). For example, an FD oracle attack on Kyber-768 successfully recovered the secret key using only seven queries with no decision errors [GNNJ23]. Moreover, this attack proved effective against first-order masked implementations of Kyber, recovering the secret key with just 12 traces. Studies such as [NDJ23] and [DNG22] have further highlighted the vulnerabilities of advanced countermeasures, including combinations of shuffling and masking, as well as fifth-order masking, to FD oracle attacks on lattice-based KEMs.

Despite their high efficiency in terms of required measurements and robustness against countermeasures, FD oracle attacks have not been explored in the context of code-based cryptography, including HQC. This gap is attributed to the unique challenges posed by code-based KEMs, such as different metrics, longer code lengths, sparse secret, and more complex decryption algorithms—similar to the challenges observed with MV-PC oracles.

In this paper, we propose the first FD-style oracle-based side-channel attack on code-based KEMs, specifically on HQC. Our approach is to generalize the defining feature of FD oracles from lattice-based cryptography—their parallelism—and apply it to HQC. To this end, we define our FD oracle as one that outputs the results of n_1 Reed-Muller inner decoders in a single query. This approach is conceptually aligned with the FD oracle attack in Kyber, whose decoding can be viewed as a concatenated code with a trivial rate-1 outer code that does not modify the message.

Building on the OT-PCA attack framework [DG25], we extend the methodology by constructing offline templates (OT) specifically for the decoding outputs of individual Reed-Muller inner decoders. As our FD oracle outputs n_1 Reed-Muller decoding results simultaneously, this approach achieves exceptional efficiency in query complexity, enabling the recovery of the secret key for hqc-128 with only two queries to a perfect FD oracle, and for hqc-192 and hqc-256 with four queries. Additionally, simulations confirm the robustness of this attack, demonstrating its effectiveness even when oracle responses are imperfect.

We validated our attack on an unprotected HQC implementation running on a Chip-Whisperer platform equipped with an ARM Cortex-M4 microcontroller. The experiments demonstrated a two-trace attack across 1 000 tests, with a success probability consistent with the simulated value for a perfect FD oracle. In these experiments, the FD oracle was constructed from strong leakage sources spanning hundreds of thousands of clock cycles. For example, in hqc-128, one notable leakage source in our experiments—the syndrome

computation during Reed-Solomon decoding—requires over 2 million clock cycles across 30 iterations. Remarkably, our approach achieved a two-trace attack using only three iterations; however, the remaining iterations could prove useful when targeting protected implementations. This is in stark contrast to lattice-based KEMs, where FD oracles are typically derived from finer-grained, shorter, and weaker leakages. Nevertheless, attacks on lattice-based KEMs have shown that even these weaker leakages are potent enough to defeat countermeasures like masking in just a few traces (see e.g. [GNNJ23]).

These results have significant implications extending beyond the security of unprotected HQC implementations. They indicate that mitigating FD oracle attacks on HQC, particularly on resource-constrained platforms like the ARM Cortex-M4, may necessitate high-order masking. Such countermeasures could introduce an overhead of tens of millions of clock cycles, posing significant challenges to the feasibility of secure HQC implementations on the ARM Cortex-M4.

Summary of Contributions. The core contributions are:

- We present the first MV-PC oracle-based attack on HQC, and more broadly, the **first** such attack on code-based KEMs. This attack achieves a significant reduction in the number of required oracle calls when an MV-PC oracle is available. Furthermore, it represents a significant advancement for side-channel attacks, parallel mismatch attacks, and fault-injection attacks in code-based KEM/PKE research.
- We propose the first FD oracle-based attack tailored to HQC, which is also the **first** FD oracle attack on code-based KEMs. Our results demonstrate remarkable efficiency in query complexity, recovering the secret key for `hqc-128` with only two queries to a perfect FD oracle, and four queries for `hqc-192` and `hqc-256`. Simulations further highlight the robustness of our attack against imperfect oracle responses.
- We validate the proposed MV-PC and FD oracle-based attacks through real-world experiments on a ChipWhisperer platform, demonstrating the feasibility of constructing accurate oracles from power leakage.
- We highlight the implications of our findings for countermeasures, particularly the challenges of masking HQC on the Cortex-M4 platform against FD oracle attacks due to the long and strong leakage profile of certain operations like syndrome computation.

The source code is available at <https://github.com/ot-pca/mv-fd-oracle>.

Organization. This paper is structured as follows. Section 2 reviews the necessary background on the HQC scheme, relevant threat models, and the OT-PCA attack framework. Our novel MV-PC and FD oracle attacks are presented in Section 3. Validation of these attacks on a ChipWhisperer platform is detailed in Section 4. A discussion of the implications and limitations of our attacks is provided in Section 5. Finally, Section 6 concludes the paper.

2 Preliminaries

Notations. We define the binary field as $\mathbb{F}_2$, where the operations of addition and subtraction are identical. The polynomial ring considered is $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, with n being a positive integer. Any polynomial $h \in \mathcal{R}$ can equivalently be represented as a length- n row vector $\mathbf{h}$ over $\mathbb{F}_2$. These two notations are used interchangeably when unambiguous. The Hamming weight of a polynomial h , represented as $w_H(h)$, indicates the count of non-zero entries in the corresponding vector representation. The index set

ranging from a to $b - 1$ (for positive integers $a < b$) is denoted by $[a..b]$. We use $\leftarrow_{\$}(\mathcal{S})$ to represent uniform random sampling from a set $\mathcal{S}$, while $\leftarrow_{\$}(\mathcal{R}, w)$ refers to sampling from $\mathcal{R}$ subject to the constraint that the sampled element has a Hamming weight of w . Finally, the Bernoulli distribution Ber_p is defined such that the value 1 is assigned with probability p , and 0 with probability $1 - p$.

2.1 Hamming Quasi-Cyclic (HQC)

Hamming Quasi-Cyclic (HQC) [AAB⁺22] is a prominent code-based Key Encapsulation Mechanism (KEM) that secures communication by relying on the hardness of decoding random quasi-cyclic codes under the Hamming metric. Similar to other NIST post-quantum KEM candidates, HQC begins with an IND-CPA (Indistinguishability under Chosen Plaintext Attack) secure Public Key Encryption (PKE) scheme (HQC.CPAPKE) and then transforms it into an IND-CCA (Indistinguishability under Chosen Ciphertext Attack) secure KEM (HQC.CCAKEM). This transformation is achieved through the Hofheinz-Hövelmanns-Kiltz (HHK) approach [HHK17], which is a variant of the well-known Fujisaki-Okamoto (FO) transform [FO99].

To provide a clear understanding, we first describe the core PKE scheme used in HQC and subsequently explain the extension to a CCA-secure KEM.

2.1.1 Public Key Encryption (PKE) Core in HQC

The PKE core of HQC consists of three main components: key generation, encryption, and decryption, denoted as **KeyGen**, **Enc**, and **Dec**, respectively.

During the key generation phase (**KeyGen**), three polynomials are sampled: $h \leftarrow_{\$}(\mathcal{R})$, $x \leftarrow_{\$}(\mathcal{R}, \omega)$, and $y \leftarrow_{\$}(\mathcal{R}, \omega)$, with the condition that $w_H(x) = w_H(y) = \omega$. The secret key is defined as (x, y) , while the public key comprises $(h, s = x + h \cdot y)$.

The encryption procedure (**Enc**) begins by generating the polynomials $r_1 \leftarrow_{\$}(\mathcal{R}, \omega_r)$, $r_2 \leftarrow_{\$}(\mathcal{R}, \omega_r)$, and $e \leftarrow_{\$}(\mathcal{R}, \omega_e)$. The process can be made deterministic by initializing a pseudo-random number generator (PRNG) with a seed θ . The ciphertext (u, v) is constructed as follows:

$$u = r_1 + h \cdot r_2 \text{ and } v = m\mathbf{G} + s \cdot r_2 + e$$

where $\mathbf{G}$ denotes the generator matrix for the linear code $\mathcal{C}$, which in HQC is realized through a concatenated code consisting of Reed-Muller (RM) and Reed-Solomon (RS) codes.

Decryption (**Dec**) involves decoding using $\mathcal{C}.\text{Decode}(v - u \cdot y)$. We derive

$$v - u \cdot y = m\mathbf{G} + s \cdot r_2 + e - (r_1 + h \cdot r_2) \cdot y = m\mathbf{G} + e'$$

where $e' = x \cdot r_2 - r_1 \cdot y + e$, given $s = x + h \cdot y$. If the Hamming weight $w_H(e')$ is small enough, the decoder is able to successfully recover the original message m . In cases where the Hamming weight exceeds the decoder's threshold, decryption errors may occur.

The latest HQC specifications utilize a concatenated code based on Reed-Muller (RM) and Reed-Solomon (RS) codes. The message m is encoded into a codeword $m\mathbf{G}$ using a generator matrix $\mathbf{G}$ in the space $\mathbb{F}_2^{k_m \times n_1 n_2}$, with $k_m = 8k_1$. The encoding proceeds by first encoding the message $m \in \mathbb{F}_2^{k_1}$ to $m_1 \in \mathbb{F}_2^{n_1}$ using the outer $[n_1, k_1, d_{\text{RS}}]$ RS code. Each byte $m_{1,i}$, for $0 \leq i < n_1$, is then encoded by the inner duplicated RM code to form $\tilde{m}_{1,i} \in \mathbb{F}_2^{n_2}$. Consequently, the resulting codeword is $m\mathbf{G} = (\tilde{m}_{1,0}, \dots, \tilde{m}_{1,n_1-1})$, with a dimension of $n_1 n_2$. Any excess bits beyond $n_1 n_2$ are truncated as required.

The input to $\mathcal{C}.\text{Decode}$ is the vector $V = v - u \cdot y$, where $V \in \mathbb{F}_2^{n_1 n_2}$. The vector V is divided into n_1 distinct blocks, denoted as $V = (V_0, \dots, V_{n_1-1})$, with each block $V_i \in \mathbb{F}_2^{n_2}$. These blocks, referred to as 'inner blocks' for $0 \leq i < n_1$, undergo decoding via the inner duplicated RM decoder, resulting in outputs $c_i^{\text{RS}} \in \mathbb{F}_2^8$. These are concatenated

Table 1: HQC parameter sets [AAB⁺22]. The base code is a first-order [128, 8, 64] Reed-Muller code. By duplicating each bit, we construct duplicated Reed-Muller codes with parameters [384, 8, 192] (3 duplications) and [640, 8, 320] (5 duplications).

Instance	RS-S			Duplicated RM		n	ω	$\omega_r = \omega_e$
	n_1	k_1	d_{RS}	n_2	d_{RM}			
hqc-128	46	16	31	384	192	17669	66	75
hqc-192	56	24	33	640	320	35851	100	114
hqc-256	90	32	59	640	320	57637	131	149

to form a sequence of $8n_1$ bits, represented as $c^{\text{RS}} = (c_0^{\text{RS}}, \dots, c_{n_1-1}^{\text{RS}})$. This sequence c^{RS} , a noisy codeword of the external RS code, is then decoded into k_1 elements in $\mathbb{F}_{256}$, which correspond to the original message of k_1 bytes.

The three HQC parameter sets, hqc-128, hqc-192, and hqc-256, defined in [AAB⁺22], are presented in Table 1.

2.1.2 Key Encapsulation Mechanism (KEM) in HQC

The enhanced KEM variant of HQC extends its PKE core by integrating two essential hash functions, $\mathcal{G}$ and $\mathcal{K}$. To construct the IND-CCA secure KEM, referred to as HQC.CCAKEM, the process is illustrated in Figure 1. This construction achieves de-randomization of the encryption process by using a seed θ , which is derived from hashing the message m , the public key, and a randomly chosen 128-bit salt. In the decapsulation stage, a re-encryption step is used to confirm the authenticity of the received ciphertext.

Input: pk

Output: $K, c = (u, v)$, salt

- 1: $m \leftarrow \$(\mathbb{F}_2^{k_m})$
- 2: salt $\leftarrow \$(\mathbb{F}_2^{128})$
- 3: $\theta \leftarrow \mathcal{G}(m || pk || \text{salt})$
- 4: $c \leftarrow \text{PKE.Encrypt}(pk, m, \theta)$
- 5: $K \leftarrow \mathcal{K}(m, c)$

(a) KEM.Encaps.

Input: $sk = (x, y)$, $c = (u, v)$, salt

Output: K

- 1: $m' \leftarrow \text{PKE.Decrypt}(sk, c)$
- 2: $\theta' \leftarrow \mathcal{G}(m' || pk || \text{salt})$
- 3: $c' \leftarrow \text{PKE.Encrypt}(pk, m', \theta')$
- 4: **if** $m' = \perp \vee c \neq c'$ **then**
- 5: $K \leftarrow \mathcal{K}(\sigma, c)$
- 6: **else**
- 7: $K \leftarrow \mathcal{K}(m', c)$
- 8: **end if**

(b) KEM.Decaps.

Figure 1: HQC.CCAKEM [AAB⁺22].

2.2 Threat Model

This section details the threat models underlying our analysis of adaptive oracle-based key-recovery attacks against HQC KEM implementations. We analyze two primary oracle types: multi-value plaintext-checking (MV-PC) oracles and full decryption (FD) oracles, differentiating their characteristics within the HQC context. Since a complete attack involves first creating an oracle from physical leakage and then using it to recover the secret key, our threat model details both the abstract oracle definitions and their practical construction from side-channel information.

2.2.1 Oracle Types and Capabilities

We define the **plaintext-checking (PC)** oracle, building upon existing definitions [OP01, ABP15, BDH⁺19]. A PC oracle takes as input a ciphertext c and a target plaintext m^* ,

returning 1 if c decrypts to m^* and 0 otherwise. By carefully selecting ciphertexts and observing oracle responses, an attacker can infer information about the secret key.

The **multi-value plaintext-checking (MV-PC)** oracle [RRD⁺23, TUX⁺23], denoted $\mathcal{O}_{\text{MV-PC}}$, provides enhanced capabilities. It takes as input a ciphertext c and a set of k predefined plaintexts $\{m_1^*, \dots, m_k^*\}$, returning the index i such that c decrypts to m_i^* in the set. This significantly enhances key space probing by simultaneously testing multiple plaintext candidates per oracle call.

The **full decryption (FD)** oracle represents the most powerful attack scenario. In lattice-based cryptography [XPRO20, NDGJ21, REB⁺22, NDJ23, GNNJ23], an FD oracle directly returns the complete plaintext for a given ciphertext c . This can yield hundreds of bits per query, making attacks exceptionally powerful. However, constructing an FD oracle is challenging, requiring a highly parallel leakage source that reveals independent plaintext chunks concurrently. For HQC, we define the FD oracle as returning the outputs of n_1 Reed-Muller inner decoders per query, exploiting HQC's inherent parallelism to maximize information gain. This mirrors the parallel nature of FD oracles in lattice-based schemes, where the oracle returns independent parts of the plaintext. Considering Kyber's decoding as a concatenated code with a trivial rate-1 outer code offers a unifying perspective on FD oracles across the two different KEMs.

To model real-world imperfections, consistent with previous PCO research, an accuracy parameter ρ is defined for both oracle types. An oracle with accuracy ρ returns the correct result with probability ρ and an incorrect result with probability $1 - \rho$. We denote the FD and MV-PC oracles with accuracy ρ as $\mathcal{O}_{\text{FD}}^\rho(\cdot)$ and $\mathcal{O}_{\text{MV-PC}}^\rho(\cdot)$, respectively. We model an incorrect response as a uniformly random selection from the set of possible outputs. It should be noted that for low accuracy levels, a more precise oracle can be constructed by applying techniques like majority voting over multiple oracle queries.

2.2.2 Oracle Construction and Attack Models

Our side-channel key-recovery attack model focuses on the HQC KEM decapsulation process. We assume an adversary capable of interacting with the target device by providing carefully crafted ciphertexts c to the decapsulation algorithm, exploiting side-channel leakage to construct MV-PC and FD oracles.

Our experimental evaluation utilizes an ARM Cortex-M4 platform. For this evaluation, we adopt a profiled attack model, assuming the adversary has access to a cloning device that is identical to the target. It is important to note, however, that this is a condition of our experimental setup, not a fundamental limitation of the attack methodology. This is analogous to MV-PC and FD attacks in lattice-based cryptography, where cross-device attacks with no cloning device have also been widely studied (e.g., [NDGJ21, GNNJ23]).

The MV-PC oracle attack is also relevant to parallel key mismatch and fault injection attacks. Drawing from methodologies explored in lattice-based cryptography [SLZ24, GMÅ24, MKB⁺24], we now detail the MV-PC oracle's construction using these techniques. In these attacks, the adversary crafts ciphertexts such that the decrypted message m' is restricted to a small candidate set. This ciphertext is then fed to the decapsulation algorithm. By analyzing the decapsulation output of a CPA-secure KEM [SLZ24] or introducing faults during the Fujisaki-Okamoto transform's ciphertext comparison stage (e.g., using methods similar to [XIU⁺21] or Rowhammer-based faults [MKB⁺24]) for a CCA-secure KEM, the adversary knows $\mathcal{K}(m', \text{aux})$, where aux represents auxiliary data and $\mathcal{K}$ is a hash function. The adversary then iterates through the candidate set, computing $\mathcal{K}$ for each candidate to identify m' .

2.3 Overview of the OT-PCA Attack

The OT-PCA method, introduced in [DG25], significantly reduces the oracle query complexity of PC oracle-based attacks against HQC. It uses the publicly available decoding function $\mathcal{C}.\text{Decode}(\cdot)$ of the concatenated code used in HQC to construct offline templates for extracting soft information about the secret key with fewer queries. This section provides a concise description of the OT-PCA method; interested readers are referred to [DG25] for a detailed analysis.

The Basic Settings of the OT-PCA Method. The attacker in the OT-PCA method [DG25] utilizes the publicly available decoding function $\mathcal{C}.\text{Decode}(\cdot)$ to manipulate ciphertexts and determine specific characteristics of the secret keys offline, thereby significantly reducing the number of online PC oracle queries needed. This strategy was introduced by Huang et al. [HSC⁺23] and later refined in [SGG24].

Specifically, to ascertain information about the secret key's component y , the attacker can set r_1 to a polynomial with a Hamming weight of one, denoted as X^k with k an integer, and setting r_2 to zero. X^k enables the attacker to examine different segments of the secret key y when shifted by k positions. This gives the ciphertext components as $u = X^k$ and $v = m\mathbf{G} + e$, and the input to the decoding function $\mathcal{C}.\text{Decode}(\cdot)$ becomes

$$v - u \cdot y = m\mathbf{G} + e - X^k \cdot y. \quad (1)$$

Similarly, to extract information about the secret key's component x , the attacker can set r_2 to X^k and r_1 to zero, leading to ciphertext components $u = X^k \cdot h$ and $v = m\mathbf{G} + s \cdot X^k + e$. In this case, the input to the decoding function $\mathcal{C}.\text{Decode}(\cdot)$ becomes

$$v - u \cdot y = m\mathbf{G} + s \cdot X^k + e - X^k \cdot h \cdot y = m\mathbf{G} + x \cdot X^k + e, \quad (2)$$

given the relationship $s = x + h \cdot y$.

By strategically manipulating r_1 , r_2 , and e , attackers can extract information about one length- n_2 block of the secret polynomials x and y . To achieve this, they design specific forms for e , ensuring that $\delta = \lfloor \frac{d_{RS}}{2} \rfloor$ blocks are flipped to all ones. This manipulation ensures that the outer Reed-Solomon decoding depends on the output of the target Reed-Muller block. The decoding output of the target Reed-Muller block is sensitive to specific positions within x and y , as these act as perturbations to the vector e .

Optimizing Error Patterns. To maximize the information gained from each oracle query, the OT-PCA method selects error patterns that maximize the entropy of the decryption outcomes. The method also chooses error patterns with maximal Hamming distances between them to ensure the independence of the information gathered from each query. Genetic-style algorithms are designed in [DG25] to achieve these optimization goals.

Constructing Offline Templates. The key component of the OT-PCA method [DG25] is constructing offline templates that can estimate and rank the probability that a particular secret position is one, conditioned on the observed decryption output from a PC oracle.

Offline templates $\mathcal{T}_{\epsilon, m}$ are constructed by simulating the decoding process for a list of optimized error patterns ϵ and a fixed message m . The steps are as follows:

1. Generate a large number of random vectors ξ of length n_2 , sampled according to the secret key distribution.
2. For each ξ , compute $m\mathbf{G}_{RM} + \epsilon + \xi$, where $\mathbf{G}_{RM}$ is the generator matrix of the Reed-Muller code.

3. Input the result into the Reed-Muller decoder to obtain the decoding outcome o (either *Success* or *Failure*).
4. Estimate the empirical conditional probability $\Pr(\xi_i = 1 \mid o)$ for each position i .

These conditional probabilities form the offline template $\mathcal{T}_{\epsilon, m}$, which provides soft information about the likelihood of each bit being 1, given the decryption outcome.

Online Attack Phase. In the online phase, the OT-PCA method crafts ciphertexts using the optimized t error patterns and sends them to the decryption oracle to obtain the decryption outcomes. Using the offline templates and the collected decryption results $\mathbf{o} = (o_1, o_2, \dots, o_t)$, the OT-PCA method estimates the posterior probabilities for each bit z_i of the secret key vector z (formed by concatenating the secret polynomials x and y):

$$\Pr(z_i = 1 \mid \mathbf{o}) \propto \prod_{j=1}^t \Pr(z_i = 1 \mid o_j), \quad (3)$$

where $\Pr(z_i = 1 \mid o_j)$ is obtained from the offline templates $\mathcal{T}_{\epsilon_j, m}$. The OT-PCA method then ranks these estimated probabilities, identifying the positions most likely to be zero by evaluating the right-hand side of Equation (3).

Full Key Recovery Using ISD with Soft Information. The OT-PCA method employs a modified Information Set Decoding (ISD) algorithm that incorporates the soft information obtained during the online phase. This enhancement improves key recovery efficiency.

The public key $s = x + h \cdot y$ can be expressed in matrix form:

$$\begin{bmatrix} \mathbf{I}_n & \text{rot}(h) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = s, \quad (4)$$

where $\mathbf{I}_n$ represents the identity matrix and $\text{rot}(h)$ denotes the circulant matrix generated by h . The post-processing method of OT-PCA builds upon the Finiasz and Sendrier ISD framework [FS09]. This framework involves selecting a permutation of positions and transforming the matrix $\begin{bmatrix} \mathbf{I}_n & \text{rot}(h) \end{bmatrix}$ into a quasi-standard form:

$$\tilde{\mathbf{H}} := \begin{pmatrix} \mathbf{Q}' & \mathbf{0} \\ \mathbf{Q}'' & \mathbf{I}_{n-\ell} \end{pmatrix}, \quad (5)$$

where $\mathbf{Q}'$ is a $\ell \times (n + \ell)$ matrix, $\mathbf{Q}''$ is a $(n - \ell) \times (n + \ell)$ matrix, and $\mathbf{0}$ is a $\ell \times (n - \ell)$ zero matrix. Instead of randomly selecting $n + \ell$ positions for matrix $\mathbf{Q}'$, the OT-PCA method leverages the soft information derived from the online decryption oracle calls. Specifically, it reorders the index set $[0..2n_1n_2]$ based on the probability of each secret bit being 1, selecting the top $n + \ell$ positions with the smallest probabilities.

To further refine the selection process and improve efficiency, two algorithm parameters, T_0 and T_1 , are introduced. From the T_0 positions with the lowest probabilities of being 1, T_1 positions are selected and assumed to be zero. These positions are then removed from $\tilde{\mathbf{H}}$. The remaining index set after removing these T_1 positions is denoted as $\mathcal{I}$.

The modified ISD algorithm employs a collision procedure that divides the index set $\mathcal{I}$ into two equal-sized subsets, $\mathcal{I}_A$ and $\mathcal{I}_B$. Central to this search is the parameter w_0 , which represents the assumed maximum number of errors (i.e., non-zero entries in the secret key) within each of the two subsets. The algorithm generates two lists enumerating all possible error patterns of length $\frac{n+\ell-T_1}{2}$ with a Hamming weight up to w_0 and searches for a collision that solves the system. Syndrome information is incorporated into one list to identify collisions that satisfy the partial syndrome of ℓ positions within the Finiasz-Sendrier framework.

The post-processing stage involves a one-time Gaussian elimination and n_{iter} inner rounds of the collision search (with n_{iter} set to 16 by default). The dominant complexity factor is thus $\max\{C_{\text{Gauss}}, n_{iter} \cdot C_{\text{list}}\}$. The complexity of generating the lists for each round is given by:

$$C_{\text{list}} = 2w_0 \cdot \ell \cdot \binom{\frac{n+\ell-T_1}{2}}{w_0}. \quad (6)$$

To maintain computational efficiency, w_0 is limited to small values, typically 2 or 3. When w_0 is set to 3, the added cost of the collision search is comparable to Gaussian elimination; when w_0 is set to 2, this cost is significantly lower. The collision procedure outputs potential candidates that must be verified against Equation (4); ℓ is chosen to be sufficiently large to make this verification step negligible.

The algorithm succeeds if, during one of the n_{iter} inner rounds, both of the following conditions are met: 1) the T_1 positions assumed to be zero are indeed error-free, and 2) the actual number of errors in the positions within $\mathcal{I}_A$ and $\mathcal{I}_B$ are both no more than w_0 . This success probability is denoted as P_{success} . Following the setting in [DG25], the number of error patterns, t , used in the online phase is chosen such that for a post-processing search with $w_0 = 3$, the simulated success probability P_{success} is greater than 50%.

Unless specified otherwise, new attacks use the post-processing parameters from [DG25], where $\ell = 51$, $T_0 = 12\,000$, $T_1 = 10\,000$, and $n_{iter} = 16$ for hqc-128. The post-processing complexity analysis remains unchanged with these parameters.¹

3 Method: New MV-PC and FD Attacks

This section introduces two novel oracle-based attacks targeting the HQC KEM: the multi-value plaintext-checking (MV-PC) attack and the full decryption (FD) attack. Both attacks are built upon the OT-PCA method [DG25], adapting and extending its core methodology—namely: 1) generation of chosen ciphertexts, 2) optimization of error patterns, 3) construction of offline templates (OT), and 4) online acquisition of soft information for subsequent Information Set Decoding (ISD)—to exploit the distinct characteristics of the MV-PC and FD oracles. Sections 3.1 and 3.2 detail the attack methodologies and present the simulation results for the MV-PC and FD attacks, respectively. Section 3.3 then summarizes the distinction between these new methodologies and the OT-PCA approach. The primary analysis focuses on the HQC parameter set hqc-128, with results for hqc-192 and hqc-256 presented in Appendix A.

For the proposed attacks, the offline preparation phase is not specific to any particular attack instance. Therefore, we exclude the cost of this offline phase from our overall attack cost estimation. OTs are constructed exclusively using a perfect oracle.

3.1 An MV-PC Oracle Attack

This section introduces a novel chosen ciphertext attack on HQC utilizing an MV-PC oracle. The attack assumes the availability of an MV-PC oracle capable of recovering the decryption output from a predefined small set of messages. We outline the attack methodology and present the corresponding simulation results.

¹In [DG25], with the suggested attack parameters, the list size for the two lists used in the collision search and the added post-processing cost are reported as follows: for hqc-128, these are 2^{23} and 2^{34} for $w_0 = 2$, and 2^{33} and 2^{45} for $w_0 = 3$, respectively; for hqc-192, these values are 2^{25} and 2^{37} for $w_0 = 2$, and 2^{36} and 2^{49} for $w_0 = 3$, respectively; and for hqc-256, they are 2^{27} and 2^{38} for $w_0 = 2$, and 2^{39} and 2^{51} for $w_0 = 3$, respectively.

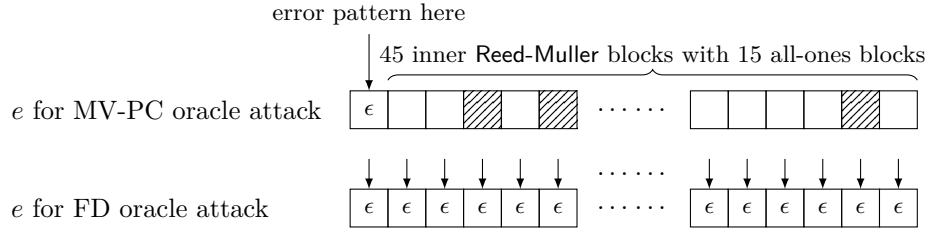


Figure 2: Illustration of vector e for MV-PC and FD attacks for hqc-128.

3.1.1 The Attack Methodology

Our ciphertext selection method closely resembles the OT-PCA approach [DG25]. As in the OT-PCA approach, an attacker seeking information about the secret key component y can set r_1 to a polynomial with a Hamming weight of one, represented as X^k , where k is an integer. Concurrently, they set r_2 to zero. By varying k , the attacker effectively probes different parts of the secret key y . To extract information about the secret key component x , the attacker sets r_2 to X^k and r_1 to zero. This results in ciphertext components $u = X^k \cdot h$ and $v = m\mathbf{G} + s \cdot X^k + e$. We set m to be 0 by default. As shown in Equations (1) and (2), with this setting the partial key $x \cdot X^k$ or $y \cdot X^k$ can be regarded as a minor perturbation introduced to e .

Similar to the OT-PCA approach [DG25], the MV-PC attack optimizes a length- n_2 error pattern ϵ positioned within the initial segment of e . However, our MV-PC method imposes distinct properties on the vector e as we seek to obtain multiple decryption outputs with it instead of the binary outputs, ‘success’ or ‘failure’, in the OT-PCA approach. To generate multiple outputs with minor perturbations introduced by the partial secret key, it is essential to operate near the decoding boundary of the concatenated codes. Our method accomplishes this by controlling one inner Reed-Muller block and fixing $\delta = \lfloor \frac{d_{RS}}{2} \rfloor$ (15 for hqc-128) blocks to all ones, while setting the remaining blocks to zero. Experimental results demonstrate that placing a length- n_2 error pattern in the first Reed-Muller block, combined with a small perturbation, leads to different decryption outcomes. The upper row of blocks in Figure 2 illustrates how the vector e for MV-PC oracle attack is configured for hqc-128.²

A simple genetic algorithm with random starts is developed to identify a length- n_2 pattern to insert into the initial segment of e with the goal of maximizing the entropy of the decryption outputs. In each generation, the algorithm randomly samples multiple mutations by flipping one or two bits of the current best pattern. For each mutation, the decryption is executed a specified number of times to obtain the entropy of its output distribution. The entropy is calculated as $H = -\sum_{d \in \text{DO}} p_d \log_2(p_d)$, where p_d is the empirical probability of decryption output d and DO denotes the set of all decryption outputs obtained. The mutation with the highest entropy among those exceeding that of the current best pattern is retained for the next generation. Through this optimization process, we obtained error patterns with entropy values exceeding 1.9 bits. The best performing error patterns are thus heuristically found and we did not observe a specific structure among them. Furthermore, for each of these patterns, the optimized entropy value corresponds to the amount of information that the oracle reveals.

Multi-Value Offline Templates for Decryption Outcomes. After optimization, we select t length- n_2 patterns (denoted $\mathcal{E} = \{\epsilon_1, \epsilon_2, \dots, \epsilon_t\}$). For a given error pattern ϵ , we construct templates $\mathcal{T}_{\epsilon, m}$ as follows. We sample 5 000 000 length- n_2 sparse vectors ξ from the secret

²The fifteen all-ones blocks are randomly chosen but fixed for finding the error patterns and in our subsequent analysis. They have indices [3, 5, 9, 10, 13, 14, 15, 17, 23, 25, 28, 31, 35, 39, 44] for hqc-128.

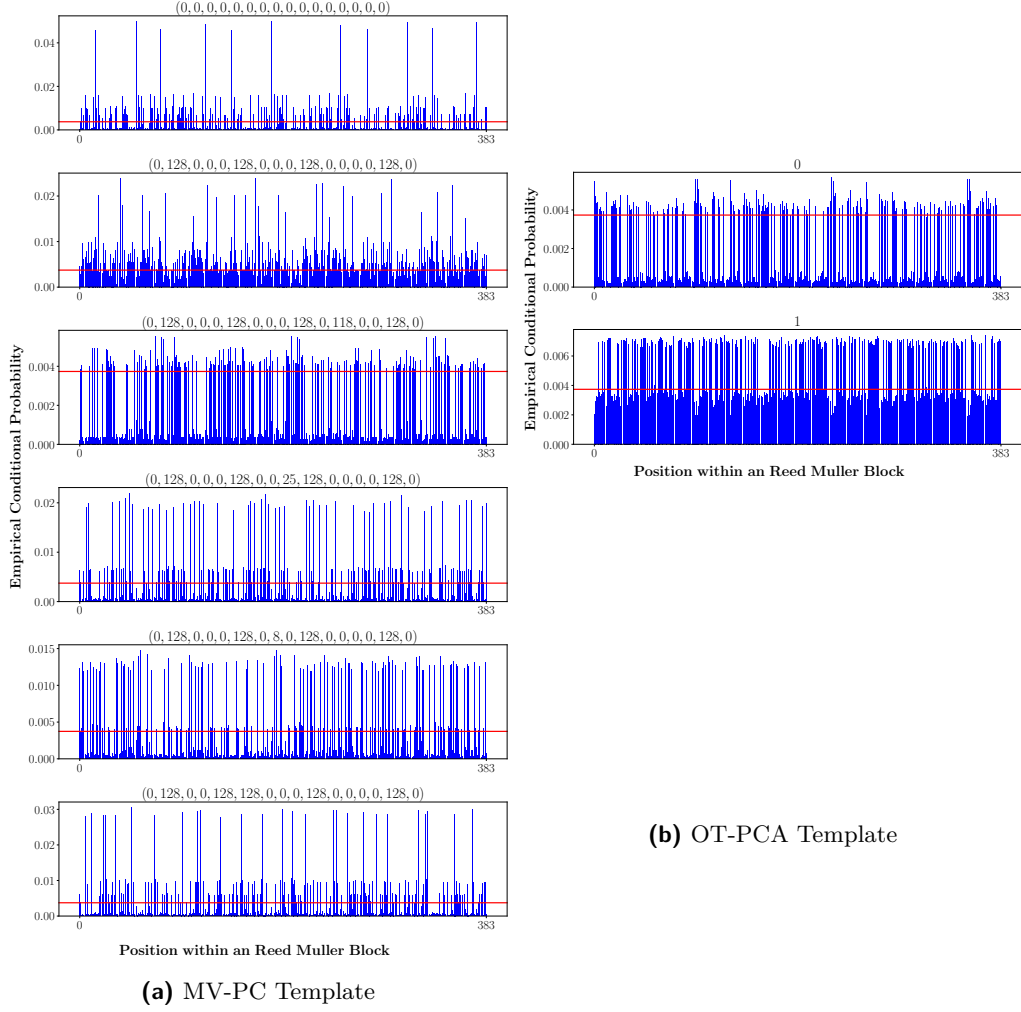


Figure 3: Comparison of MV-PC and OT-PCA templates for hqc-128 . The left panel illustrates the offline template $\mathcal{T}_{\epsilon,m}$ for a selected error pattern in the MV-PC method, which produces six distinct decoding outputs (shown as 16-integer tuples above each plot). The right panel shows the template for one selected error pattern in the OT-PCA method, which results in two decoding outputs, 0 (success) and 1 (failure). The y -axis represents the empirical conditional probability (posterior probability), while the red horizontal line indicates the a-priori probability of a bit being one within a Reed-Muller block, which depends on HQC’s parameters and equals $\frac{\xi}{n}$ (approximately 3.74×10^{-3} for hqc-128).

key distribution. For each ξ , we perform an XOR operation with the error pattern ϵ and position the resulting length- n_2 vector as the first block of a codeword of length $n_1 n_2$. Subsequently, we set 15 specific blocks in the remaining segments to all ones, while keeping the other blocks as all zeros. We then query the concatenated Reed-Muller and Reed-Solomon decoders with this codeword to obtain an output message. The 5 000 000 vectors ξ are categorized based on the decoder’s output. For each category, we aggregate the vectors to compute the empirical probability of each bit in ξ being one. The resulting Offline Templates (OT) consist of multiple length- n_2 vectors representing these empirical probabilities, with one vector corresponding to each possible MV-PC oracle output.

Figure 3 illustrates the offline templates, $\mathcal{T}_{\epsilon,m}$, for hqc-128 in the MV-PC and OT-PCA

methods. The left panel displays the template constructed for a selected error pattern in the MV-PC method. This error pattern, when combined with secret (sub)-key patterns, can lead to six different decryption outputs, as indicated by the tuples of 16 integers above each plot. This error pattern thus effectively divides the key space into six partitions. In comparison, the right panel shows the template in the OT-PCA method, where a selected error pattern yields only two decoding outputs, 0 (success) and 1 (failure).

Online Attack Phase and Post-processing Phase. Following the OT-PCA framework [DG25], the remaining process consists of an online attack phase and a subsequent post-processing phase. The online attack phase adheres to the pseudocode outlined in Algorithm 1. Post-processing leverages the ISD procedure with soft information as detailed in Section 2.3.

Algorithm 1 New MV-PC Oracle Based Attack on HQC (Online Phase)

Input: Public key element h , message m (default $m = 0$), salt $salt$
Set of t error patterns $\mathcal{E} = \{\epsilon_1, \epsilon_2, \dots, \epsilon_t\}$
Pre-calculated offline templates $\mathcal{T}_{\epsilon, m}$ for message m and each pattern $\epsilon \in \mathcal{E}$
Scheme parameters $(n_1, n_2, d_{RS}, \mathbf{G})$

Output: Reliability information matrix R

- 1: Initialize reliability scores $R_{i,j} \leftarrow 1.0$ for all $i \in \{“X”, “Y”\}$ and $j \in [0..n_1 n_2]$
- 2: **for** $i_{tmp} \in [0..n_1]$ **do**
- 3: Set shift $k \leftarrow -n_2 \cdot i_{tmp}$ $\triangleright$ Calculate the specific offset
- 4: **for** $i \in \{“X”, “Y”\}$ **do** $\triangleright$ Iterate through target secret key component type
- 5: **for** each pattern ϵ in $\mathcal{E}$ **do** $\triangleright$ Loop over available error patterns
- 6: Configure the error vector e :
- 7: Flip bits in $\delta = \lfloor d_{RS}/2 \rfloor$ Reed-Muller code blocks (excluding the first block).
- 8: Set the first block of e according to the current pattern ϵ .
- 9: **if** $i = “X”$ **then** $\triangleright$ Targeting secret key component x
- 10: Set $r_2 \leftarrow X^k$
- 11: Set $u \leftarrow X^k \cdot h$
- 12: **else if** $i = “Y”$ **then** $\triangleright$ Targeting secret key component y
- 13: Set $r_2 \leftarrow 0$
- 14: Set $u \leftarrow X^k$
- 15: **end if**
- 16: Form the ciphertext $c \leftarrow (u, m\mathbf{G} + s \cdot r_2 + e, salt)$
- 17: Obtain oracle output: $\text{OutputMVPC} \leftarrow \mathcal{O}_{\text{MVPC}}^p(c)$
- 18: **for** $j \in [0..n_2]$ **do** $\triangleright$ Loop over positions within a block
- 19: Fetch probability $P \leftarrow \Pr(\xi_j = 1 | \text{OutputMVPC})$ from template $\mathcal{T}_{\epsilon, m}$
- 20: Update reliability: $R_{i,j-k} \leftarrow R_{i,j-k} \times P$
- 21: **end for**
- 22: **end for**
- 23: **end for**
- 24: **end for**
- 25: **return** Final reliability scores $R_{i,j}$ for $i \in \{“X”, “Y”\}, j \in [0..n_1 n_2]$

During the online attack phase, chosen ciphertexts are fed to the MV-PC oracle to obtain decryption outcomes. We define an integer $k = -n_2 \cdot i$ for each i in the range $[0..n_1]$. For hqc-128, the Reed-Muller block size n_2 is 384. Iterating i from 0 to $n_1 - 1$ in the calculation $k = -n_2 \cdot i$ effectively shifts each Reed-Muller block of the secret key into the first block’s position. For each shift, t measurements of the n_2 secret values within that block are collected. After completing all n_1 shifts, soft information for the first $n_1 \cdot n_2$

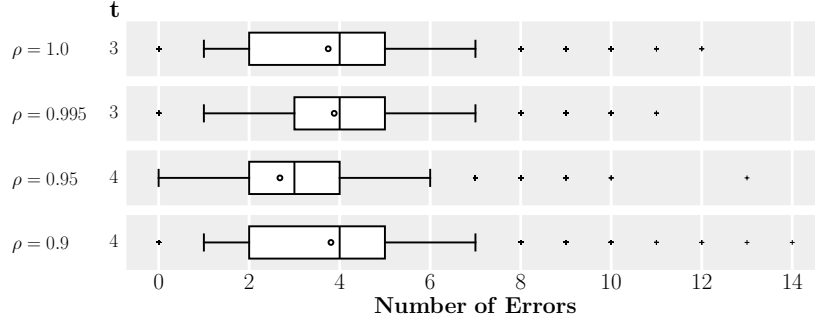


Figure 4: Distributions of the number of errors in the initial n positions of the reordered z vector across different combinations of oracle accuracy level (ρ) and the number of MV-PC oracle calls ($2t \cdot n_1$) for hqc-128. Each scenario is evaluated using 10 000 randomly generated keys. Whiskers indicate the 5th and 95th percentile ranges, the small circles mark the average error counts, and the vertical lines within the box indicate the median values.

Table 2: Simulated success rates P_{success} for hqc-128 given different combinations of oracle accuracy (ρ) and the number of MV-PC oracle calls ($2t \cdot n_1$). w_0 is defined in Section 2.3. P_{success} in each scenario is based on 10 000 random keys.

ρ	t	P_{success}	
		$w_0 = 2$	$w_0 = 3$
1	3	49.23%	63.39%
0.995	3	47.39%	61.33%
0.95	4	67.24%	75.84%
0.9	4	46.76%	59.02%

secret entries of both x and y is obtained.

Concatenating these $2n_1 \cdot n_2$ entries yields a new vector z . Consequently, the total number of required online oracle calls for the MV-PC process amounts to $2t \cdot n_1$. With the offline template established, we rank the conditional probabilities $\Pr(z_i = 1 \mid \mathbf{o})$ by calculating the product: $\prod_{j=1}^t \Pr(z_i = 1 \mid o_j)$, for $0 \leq i < 2n_1n_2$, where the values $\Pr(z_i = 1 \mid o_j)$ are derived from the OTs.

3.1.2 Simulation Results

We present simulation results using a MV-PC oracle. We assess the method’s performance with 10 000 randomly selected keys, evaluating various accuracy levels and different numbers of oracle calls. Consistent with prior analyses, our focus remains on hqc-128, and the simulations use offline templates built with a perfect oracle. We adopted the same parameters as [DG25] to ensure identical post-processing costs.

Figure 4 illustrates the simulated number of errors given different oracle accuracies (ρ) and numbers of selected patterns (t). The number of errors is defined as the count of missed ones in positions expected to be zero in the initial n positions of the reordered z vector. To obtain the total MV-PC oracle calls, a scaling factor of $2n_1$ (92 for hqc-128) is applied to the number of selected patterns (t). Table 2 details the corresponding simulated success rates for full key recovery (P_{success}), which is defined in Section 2.3. Notably, with a perfectly accurate oracle, only 276 MV-PC oracle calls are required for key recovery with P_{success} larger than 50%, compared to 460 PC oracle calls needed by the OT-PCA method [DG25].

3.2 An FD Oracle Attack

We next present the first FD-style oracle attacks targeting the HQC KEM. Our approach assumes an FD oracle that outputs the results of n_1 Reed-Muller inner decoders from a single query. We describe the methodology in detail and then provide simulation results.

3.2.1 Attack Methodology

Our ciphertext selection approach is analogous to the OT-PCA method [DG25], with a primary distinction: the omission of shifting through multiplication by X^k . This shifting is essential in OT-PCA, as an attacker needs to construct a different ciphertext (by varying k) for each length- n_2 segment of the secret key. In contrast, the FD oracle simultaneously outputs decoding results for n_1 Reed-Muller decodings, thus revealing information for $n_1 \cdot n_2$ secret positions in one single query. Consequently, this shifting is unnecessary, which significantly simplifies the ciphertext construction.

To extract information about the secret key component y , an attacker sets $r_1 = 1$ (equivalent to the polynomial X^0) and $r_2 = 0$. Conversely, to target the secret key component x , the attacker sets $r_2 = 1$ and $r_1 = 0$. This configuration results in ciphertext components $u = h$ and $v = m\mathbf{G} + s + e$, with m set to 0 by default.

The lower row of blocks in Figure 2 illustrates how the vector e for FD oracle attack is configured. Essentially, the same error pattern ϵ is placed in each of the 46 inner Reed-Muller blocks.

To find error patterns for our FD attack, a genetic algorithm similar to that in Section ??_optimizationsec:theory_optimizationopted. The difference is that the goal is to maximize the entropy of the Reed-Muller decoding outputs. Through the optimization process, we obtained error patterns with entropy values exceeding 2.8 bits, surpassing the 1.9 bits achieved by the MV-PC attack.

Additionally, the FD attack leverages the ability to apply OTs independently across $n_1 = 46$ Reed-Muller blocks for hqc-128. This high degree of parallelism results in a significantly larger secret information leakage per oracle query, totaling approximately 128 bits (46×2.8 bits) for hqc-128. These enhancements lead to a dramatic reduction in query complexity.

Multi-Value Offline Templates for Reed-Muller Decoding Outcomes. We now describe the construction of offline templates $\mathcal{T}_{\epsilon,m}$ for a given error pattern ϵ and message m . Given ϵ , we generate 10 000 000 low-weight vectors ξ of length n_2 , sampled according to their frequency in the secret key distribution. We categorize each ξ based on its decoding output o from the decoder function. Categories with fewer than 5 000 samples are discarded, as they represent rare decoding outputs (occurring with probability less than $\frac{1}{2000}$). For each remaining category, we aggregate the ξ vectors to obtain an empirical probability for each entry in ξ , indicating the probability of the entry being one. The resulting OTs consist of multiple length- n_2 vectors representing these empirical probabilities, with one vector corresponding to each possible Reed-Muller decoding output. The OTs exhibit similar characteristics to the MV-PC templates in Figure 3 but contain more length- n_2 vectors. For instance, the selected error pattern for hqc-128 can lead to 27 distinct decoding outputs, thus 27 vectors of empirical probabilities.

During the attack phase, given a Reed-Muller decoding output, we locate the corresponding length- n_2 vector and infer the probability of each position being one. If the decoding output is absent from the templates, we randomly assign a vector, which introduces inaccuracy in the attack.

Online Attack Phase and Post-processing Phase. The remaining of our FD oracle attack comprises an online attack phase and a post-processing phase. Algorithm ??_-

attack_onlinealg:fd_attack_onlinestrates the online attack phase. We adopt the ISD procedure with soft information derived from OT-PCA, as detailed in Section 2.3.

During the online phase, we input t selected ciphertexts, corresponding to the t determined patterns of length n_2 identified during offline optimization (denoted $\mathcal{E} = \{\epsilon_1, \epsilon_2, \dots, \epsilon_t\}$), to the constructed FD oracle. This yields $t \cdot n_1$ Reed-Muller decoding outputs, denoted by OutputFD_k^j for $k \in [0..n_1]$ and $j \in [1..t + 1]$.

Similar to OT-PCA, we use Equation (3) to rank the probability of each bit x_i (or y_i) in the secret key. This involves calculating the product $\prod_{j=1}^t \Pr(x_i = 1 \mid \text{OutputFD}_k^j)$, where $k = \lfloor \frac{i}{n_2} \rfloor$ and $\Pr(x_i = 1 \mid \text{OutputFD}_k^j)$ is derived from the offline templates $\mathcal{T}_{\epsilon_j, m}$.

Since we simultaneously obtain n_1 Reed-Muller decoding outputs per FD oracle call, the required number of oracle calls is only $2t$ rather than $2t \cdot n_1$ as in the OT-PCA and MV-PC attacks.

Algorithm 2 New FD Oracle Based Attack on HQC (Online Phase)

Input: Public key element h , message m (default $m = 0$), salt salt
Set of t error patterns $\mathcal{E} = \{\epsilon_1, \epsilon_2, \dots, \epsilon_t\}$
Pre-calculated offline templates $\mathcal{T}_{\epsilon, m}$ for message m and each pattern $\epsilon \in \mathcal{E}$
Scheme parameters $(n_1, n_2, \mathbf{G})$

Output: Reliability information matrix R

- 1: Initialize reliability scores $R_{i,j} \leftarrow 1.0$ for all $i \in \{“X”, “Y”\}$ and $j \in [0..n_1 n_2]$
- 2: **for** $i \in \{“X”, “Y”\}$ **do** $\triangleright$ Iterate through target secret key component type
- 3: **for** each pattern ϵ in $\mathcal{E}$ **do** $\triangleright$ Loop over available error patterns
- 4: Construct error vector e :
- 5: Set each of the first n_1 blocks of e to the pattern ϵ (length n_2).
- 6: **if** $i = “X”$ **then** $\triangleright$ Targeting secret key component x
- 7: Set $r_2 \leftarrow 1$
- 8: Set $u \leftarrow h$
- 9: **else if** $i = “Y”$ **then** $\triangleright$ Targeting secret key component y
- 10: Set $r_2 \leftarrow 0$
- 11: Set $u \leftarrow 1$
- 12: **end if**
- 13: Form the ciphertext $c \leftarrow (u, m\mathbf{G} + s \cdot r_2 + e, \text{salt})$
- 14: Obtain oracle output: $\text{OutputFD} \leftarrow \mathcal{O}_{\text{FD}}^p(c)$ $\triangleright$ FD oracle returns n_1 outputs
- 15: **for** $k \in [0..n_1]$ **do** $\triangleright$ Loop over the n_1 blocks
- 16: **for** $j \in [0..n_2]$ **do** $\triangleright$ Loop over positions within a block
- 17: Fetch probability $P \leftarrow \Pr(\xi_j = 1 \mid \text{OutputFD}_k)$ from template $\mathcal{T}_{\epsilon, m}$
- 18: Update reliability: $R_{i,j+k \times n_2} \leftarrow R_{i,j+k \times n_2} \times P$
- 19: **end for**
- 20: **end for**
- 21: **end for**
- 22: **end for**
- 23: **return** Final reliability scores $R_{i,j}$ for $i \in \{“X”, “Y”\}, j \in [0..n_1 n_2]$

3.2.2 Simulation Results

This section presents the simulation results, which assume the existence of an FD oracle derived from side-channel measurements. These oracles provide the n_1 Reed-Muller decoding outputs. We conducted extensive simulations to assess the performance of the new attack, evaluating 10 000 random keys for each scenario. To enable a thorough analysis, we concentrate on hqc-128, as it has been the main focus of prior research.

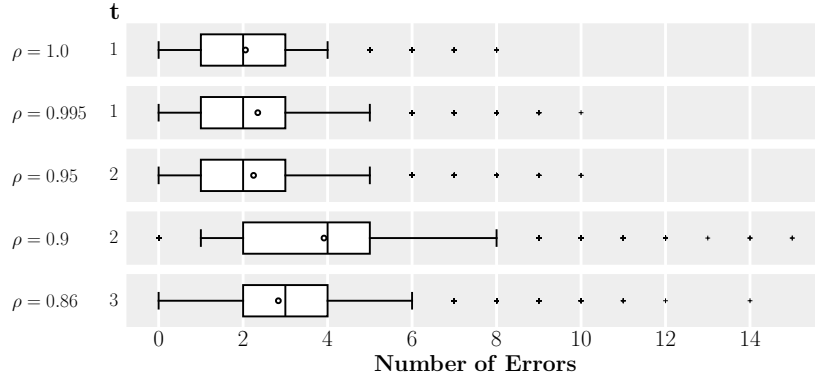


Figure 5: Distributions of the number of errors in the initial n positions of the reordered z vector across different combinations of oracle accuracy level (ρ) and the number of FD oracle calls ($2t$) for hqc-128. Each scenario is evaluated using 10 000 randomly generated keys. Whiskers indicate the 5th and 95th percentile ranges, the small circles mark the average error counts, and the vertical lines within the box indicate the median values.

Table 3: Simulated success rates P_{success} for hqc-128 given different combinations of oracle accuracy (ρ) and the number of FD oracle calls ($2t$). The parameter w_0 is defined in Section 2.3. P_{success} in each scenario is based on 10 000 random keys.

ρ	t	P_{success}	
		$w_0 = 2$	$w_0 = 3$
1	1	75.42%	78.83%
0.995	1	68.90%	74.19%
0.95	2	73.51%	78.72%
0.9	2	43.69%	54.53%
0.86	3	61.19%	68.08%

Our investigation includes a detailed performance assessment of the attack under various oracle accuracies, specifically 1, 0.995, 0.95, 0.9, and 0.86. The inclusion of 0.86 reflects the observed accuracy in our experiments, which exploit relatively weak leakage from the Reed-Muller decoding (further details are provided in Section 4.3 and Appendix C.2). By examining this range of accuracies, we can better predict our approach’s effectiveness in situations where obtaining a highly accurate PC oracle is challenging.

Figure 5 depicts the simulated error counts for various oracle accuracies (ρ) across different numbers of FD oracle calls ($2t$). Notably, the total number of FD oracle calls requires scaling by a factor of 2. Table 3 summarizes the simulated success probability for full key recovery (P_{success}), which is defined in Section 2.3. Our results demonstrate that only 2 FD oracle calls are sufficient for complete key recovery with high probability when assuming a perfect oracle. Even with a reduced oracle accuracy of 0.86, only 6 FD oracle calls are necessary to achieve full key recovery. For hqc-192 and hqc-256, four calls to the perfect FD oracle are sufficient, as shown in Table ??_moretable:fd_more

3.3 Comparison with the OT-PCA Methodology

Our MV-PC and FD attacks fundamentally enhance the OT-PCA framework [DG25] by leveraging more powerful oracles. This leads to distinct methodologies for offline template generation, error pattern optimization, and online querying, culminating in a significant reduction in attack complexity.

Oracle-Specific Templates and Error Optimization. Unlike OT-PCA, which uses a binary ‘success/failure’ oracle, our MV-PC attack employs a multi-value oracle based on the final decrypted message, while the FD attack uses one that reports the parallel outputs of individual Reed-Muller decoders. This progression allows for constructing richer offline templates and increases the information gain per query from 1 bit (OT-PCA) to 1.9 bits (MV-PC) and 2.8 bits per block (FD), for `hqc-128`. Consequently, error patterns are optimized not for a binary outcome, but to maximize the entropy of these multi-value outputs by either carefully controlling a single inner block (MV-PC) or placing the same pattern across all inner blocks (FD).

Online Query Strategy and Complexity. These enhancements culminate in a far more efficient online phase. The MV-PC attack, like OT-PCA, probes the secret key by iteratively shifting its segments, but requires fewer queries (276 vs. 460 for `hqc-128`) due to its higher information gain. Crucially, the FD attack’s inherent parallelism eliminates the need for this shifting mechanism entirely. This reduces the query complexity from $2t \cdot n_1$ to just $2t$, enabling key recovery with two queries for `hqc-128`.

4 Real-World Validation

In this section, we validate the proposed MV-PC and FD methods in real-world scenarios. Building upon the general threat model described in Section 2.2, we assume that the adversary has access to a cloning device that is identical or highly similar to the target device, enabling full control for profiling purposes.

To construct the required MV-PC and FD oracles, we adopt a deep-learning-based approach consistent with recent side-channel PCO attack research (e.g., [NDGJ21, UXT⁺22, TUX⁺23, DG25]). Specifically, we perform multi-class classification using neural networks (NNs). Our model architecture, adapted from [UXT⁺22], is detailed in Table 4. Model training is performed on a Mac Studio with a 24-core M2 Ultra CPU, a 60-core GPU, and 64 GB of RAM.

Table 4: NN hyperparameters (adapted from [UXT⁺22]).

	Input	Operator	Output	Activation Function	Batch Normalization	Pooling
FC_1	-	dense	32	SELU	Yes	No
FC_2	32	dense	32	SELU	Yes	No
FC_3	32	dense	20	SELU	Yes	No
FC_4	20	dense	20	SELU	No	No
FC_5	20	dense	-	Softmax	No	No

4.1 Experimental Setup

Our proposed methods are validated through experiments using power traces collected on the ChipWhisperer platform. We target the PQClean implementation of `hqc-128` [KSSW22] updated to the HQC 2023-04-30 submission³. Subsequent PQClean updates do not affect the code sections (i.e., encoding and decoding for the concatenated code) relevant to our experimental validations; thus, our attack remains applicable. Our experimental setup includes the ChipWhisperer Husky, the CW313 base-board, and an ATSAM4S2A microcontroller target featuring a 32-bit ARM Cortex-M4 RISC core. The system operates at a frequency of 24 MHz, with a sampling rate of 24 MS/s. The implementation is compiled using `arm-none-eabi-gcc` with the highest optimization level `-O3`.

³<https://github.com/PQClean/PQClean/pull/512>.

Table 5: Simulation results for hqc-128 with 276 MV oracle calls ($t = 3$) vs. real-world validation results with 276 traces. Real-world validation includes 1 000 tests, each with a randomly generated key pair, and employs offline templates constructed with a perfect oracle ($\rho = 1$).

	Mean Number of Errors	Success Rate (P_{success})	
		$w_0 = 2$	$w_0 = 3$
Simulation ($\rho = 1$)	3.75	49.23%	63.39%
Real-world validation	3.74	49.50%	63.00%

4.2 MV-PC Attack Validation

For the new MV-PC method, we focus on the re-encryption phase of HQC decapsulation (KEM.Decaps). Trigger functions are incorporated to capture the waveform of the PQ-CLEAN_HQC128_CLEAN_code_encode function, during which the decrypted message m' is processed to calculate the encoding $m'G$. This direct involvement of m' enables the creation of NN classifiers from power leakage.

In the validation, we selected three patterns ($t = 3$), with each pattern potentially producing 5–6 different outputs. However, decrypted messages across patterns can overlap, resulting in eleven unique m' messages in our experiments. An accurate NN classifier is essential for constructing the MV-PC oracle effectively.

4.2.1 Leakage Detection

We performed pairwise Test Vector Leakage Assessment (TVLA) on 100 power traces for each possible m' and identified strong leakage points with TVLA values exceeding 10. This analysis revealed a total of 5 607 leakage positions per trace. Detailed results are provided in Appendix B.

4.2.2 Experimental Results

We collect 3 000 traces for each of the eleven unique m' , retain the 5 607 positions with strong leakage, and split the data into training (60%), validation (20%), and testing (20%) sets. We train one model per selected pattern, achieving perfect testing accuracy for all models. These accurate NN classifiers enable the construction of reliable MV-PC oracles.

Next, we apply the trained models to traces of 1 000 newly generated random key pairs to predict decryption results from concatenated decoders. Using the predicted messages and the corresponding offline templates, we rank the conditional probabilities of secret key entries as described in Section 3.1 and calculate the success rate. Table 5 presents both simulation and real-world validation results, showing closely aligned mean errors and success rates.

4.3 FD Attack Validation

For the proposed FD method, we add trigger functions to capture the waveform during the execution of the function PQCLEAN_HQC128_CLEAN_code_decode, which handles decoding for the concatenated Reed-Muller and Reed-Solomon codes during the decryption of HQC's decapsulation KEM.Decaps phase. The function first performs Reed-Muller decoding, followed by Reed-Solomon decoding for the outer code. In the validation, we select one length-384 pattern ($t = 1$) with 27 possible Reed-Muller decoding outputs and train NN classifiers for these outputs.

4.3.1 Leakage Detection

We observe stronger leakage in the outer Reed-Solomon decoding than in Reed-Muller decoding. In HQC, Reed-Solomon decoding begins with computing syndromes from the Reed-Muller decoder outputs, which is implemented in the `compute_syndromes` function (Listing 1). The input `cdw[j]` for $j \in [0..n_1]$ represents the outputs of the n_1 Reed-Muller decoder. Recovering these values is our goal for constructing the desired FD oracle. The computation yields 2δ syndromes ($\delta = 15$ for hqc-128), indicating that leakages on the same variable occur 2δ times. We observed strong leakage patterns for `cdw[j]` where $j \in [1..n_1]$, but not for `cdw[0]`. This discrepancy can be attributed to the source code in Listing 1, specifically within the inner loop for $j \in [1..n_1]$, where `cdw[j]` serves as input to the `PQCLEAN_HQC128_CLEAN_gf_mul` function.

```

1 static void compute_syndromes(uint16_t *syndromes, uint8_t *cdw) {
2     for (size_t i = 0; i < 2 * PARAM_DELTA; ++i) {
3         for (size_t j = 1; j < PARAM_N1; ++j) {
4             syndromes[i] ^= PQCLEAN_HQC128_CLEAN_gf_mul(
5                 cdw[j],
6                 alpha_ij_pow[i][j - 1]
7             );
8         }
9         syndromes[i] ^= cdw[0];
10    }
11 }

```

Listing 1: Compute syndromes.

We measure only the first three iterations of the `compute_syndromes` function, as the leakage is already sufficiently strong. The remaining 27 iterations for hqc-128 can be used if stronger countermeasures are implemented. We start with collecting and analyzing 260 000 data points from the 781 000th timestamp of each trace, covering these three iterations. Pairwise TVLAs with 100 traces detect strong leakage (TVLA values above 7.5) in the last 45 of 46 blocks, yielding 886 data points for the first 23 blocks and 902 for the remaining 22. For `cdw[0]`, we rely on inner Reed-Muller decoding and select 3 468 data points with TVLA values above 4.5. Further details, including additional TVLA results, can be found in Appendix C.

4.3.2 Experimental Results

We construct the FD oracle using NN classifiers trained on traces from positions with strong leakage. For each block, `cdw[j]` ($j \in [0..n_1]$), we train a separate NN model to distinguish the 27 unique Reed-Muller decoding outputs for the selected error pattern ($t = 1$).

To recover the first Reed-Muller decoding block's output, we utilize 3 468 leakage points detected and collect 35 000 traces per Reed-Muller decoding output. The data are split into training (60%), validation (20%), and testing (20%) sets. After training for 100 epochs, the model achieves a test accuracy of 0.8645.

For the remaining 45 Reed-Muller decoding blocks, we use leakage points from the Reed-Solomon decoding procedure, collecting 3 000 traces per decoded output. Following the same split ratio as before, we train each model with 100 epochs, achieving a test accuracy above 0.998.

We apply the 46 trained models to traces from 1 000 newly generated random key pairs to predict the outputs of the 46 Reed-Muller decodings. Using these predictions and an offline template for the selected pattern, we rank the conditional probabilities of secret key entries. Table 6 presents both simulation and real-world validation results. The real-world validation shows slightly higher mean errors and lower success rates for $w_0 = 2$ and $w_0 = 3$, which is expected due to imperfections in the constructed oracle, particularly for the first

Table 6: Simulation results with two FD oracle calls ($t = 1$) vs. real-world validation results with two traces for `hqc-128`. Real-world validation includes 1 000 tests, each with a randomly generated key pair, and employs offline templates constructed with a perfect oracle ($\rho = 1$).

	Mean Number of Errors	Success Rate (P_{success})	
		$w_0 = 2$	$w_0 = 3$
Simulation ($\rho = 1$)	2.06	75.42%	78.83%
Real-world validation	2.32	68.10%	73.50%

Reed-Muller block (accuracy of 0.8645 on the test set). Nonetheless, the overall results align well.

4.4 Leakage Sources for Oracle Construction

Our practical validation relies on strong leakage sources identified in the PQClean implementation of HQC, where TVLA with as few as 100 traces confirms substantial leakage. These sources enable the practical construction of our proposed MV-PC and FD oracles.

For the MV-PC oracle, we found numerous strong leakage points in the encoding function, which is executed during the re-encryption phase of decapsulation. This function’s direct processing of the decrypted message m' to compute its encoding $m'G$ creates a powerful side-channel. While we focused on this source, other known leakages, such as those in PRF/PRG functions during re-encryption [UXT⁺22, TUX⁺23], could also be used to build MV-PC oracles.

For the FD oracle, we leveraged two distinct leakage sources within the main decoding function. The primary source, exhibiting substantial leakage, is the `compute_syndromes` function, specifically the `PQCLEAN_HQC128_CLEAN_gf_mul` function that processes the outputs of the inner Reed-Muller decoders. A secondary, weaker leakage source was identified in the `find_peaks` function within the inner Reed-Muller decoding procedure.

While the encoding and decoding leakages have been previously exploited in [DG25] and [SHR⁺22] respectively, their application was limited to constructing simpler binary PC oracles. Our work demonstrates that these sources can be exploited to build the more powerful MV-PC and FD oracles central to our attacks.

The ability to build oracles from various sources highlights the generic nature of our attacks. Their feasibility hinges on the existence of a suitable oracle rather than on any specific leakage characteristic. Identifying additional leakage sources or demonstrating oracle construction on different platforms and against protected implementations remains a compelling direction for future research.

5 Discussion

This section analyzes the improvements, limitations, and future directions of our proposed MV-PC and FD attacks against HQC implementations. A comparative analysis with recent related work is also presented, highlighting the unique contributions and challenges associated with our approach.

5.1 On the MV-PC Attack

Our new MV-PC oracle-based attack demonstrates substantial improvements over previous PCO methods for HQC, particularly the OT-PCA technique [DG25]. Specifically, for `hqc-128`, our approach requires only 60% of the traces needed by OT-PCA when utilizing a perfect oracle derived from a single power trace. This significant improvement stems

from enhanced exploitation of strong leakage inherent in such embedded platforms, as corroborated by other studies [RRD⁺23, TUX⁺23]. Similar gains could potentially be observed in parallel key mismatch attacks and fault-injection attacks targeting HQC, mirroring observations in other lattice-based schemes [SLZ24, MKB⁺24].

To provide a more rigorous quantitative analysis, consider `hqc-128` as an example. Our genetic algorithm optimizes information per MV-PC oracle call to exceed 1.9 bits. In contrast, OT-PCA [DG25] achieves 0.999 bits per PC oracle call with parameter $t = 5$. Roughly estimating the equivalent t for our MV-PC attack using information-theoretic principles yields: $\frac{5 \times 0.999}{1.9} \approx 3$. This estimation closely aligns with our simulation results, validating the effectiveness of the intuitive analysis. A more rigorous information-theoretical understanding represents an interesting avenue for future research.

Despite these improvements, limitations remain. While our MV-PC oracle distinguishes among 5–6 predefined messages, this falls short of the multi-class classification capabilities (up to 2^8 – 2^{12} classes) achieved by other works on unprotected ARM platforms targeting lattice-based schemes [RRD⁺23, TUX⁺23]. Furthermore, our current approach controls only one Reed-Muller code block, and attempts to extend our control to multiple Reed-Muller code blocks did not improve performance. This suggests that exploring more sophisticated methods for leveraging leakage across multiple blocks is a promising avenue for future research.

5.2 On the FD Attack

Our FD attack against HQC, while currently validated on the ARM microcontroller platform, is remarkably powerful in terms of oracle call counts (e.g., 2 for `hqc-128`, 4 for `hqc-192` and `hqc-256`). This efficiency stems from the remarkably high information extracted per oracle call (e.g., approximately 128 bits for `hqc-128`).

Despite the similarity in attack power, our FD attack differs significantly from its lattice-based counterparts. Existing lattice-based FD attacks typically rely on fine-grained leakages from individual message bits, often limited to a few clock cycles per bit [XPRO20, NDGJ21, REB⁺22, GNNJ23]. In contrast, the leakage sources exploited in our FD attack persist over substantially longer periods, potentially rendering HQC more vulnerable to side-channel attacks than previously thought.

The implications of this strong leakage in the ARM Cortex-M4 platform are particularly concerning when considering countermeasures. Even against highly protected lattice-based KEM implementations employing masking (including higher-order masking or combined countermeasures with shuffling and masking), FD oracle attacks have demonstrated remarkable success [NDGJ21, GNNJ23, NDJ23, DNG22]. For instance, an FD attack successfully compromised a first-order masked Kyber-768 implementation using only 12 traces [GNNJ23]. Given the significantly stronger leakage sources observed in the HQC KEM, achieving secure and efficient masked implementations of HQC for the ARM Cortex-M4 platform presents a substantial challenge. This warrants further investigation into the feasibility and effectiveness of higher-order masking schemes for HQC.

5.3 Countermeasures

Our MV-PC and FD oracle-based adaptive key-recovery attacks on HQC implementations present two challenges: 1) assuming an oracle, mathematically recover the secret key; 2) building such an oracle from physical leakages. The first relies on the current HQC design (e.g., concatenated codes); changing the design to resist the attack requires re-evaluating the revised scheme’s security. The focus, therefore, should be on hardening the creation of accurate MV-PC and FD oracles.

The MV-PC attack has implications for parallel key mismatch attacks and fault-injection attacks. In the former, implementers should strictly avoid key reuse in ephemeral-only

Table 7: Comparison of implementation attacks on hqc-128.

Attack	Oracle Type	Attack Type	Oracle Calls/Traces
OT-PCA [DG25]	Binary PC	Key Recovery	460
MV-PC (This work)	Multi-Value PC	Key Recovery	276
FD (This work)	Full Decryption	Key Recovery	2
Paiva et al. [PRJB23]	-	Key Recovery	2
Goy et al. [GMGL24]	-	Message Recovery	1

CPA-secure HQC KEMs. For the latter, countermeasures against traditional fault-injection attacks could be beneficial, such as using ECC-enabled RAMs to mitigate Rowhammer-based attacks or employing code-based masking to detect and correct faults.

Both MV-PC and FD oracles can be constructed from power and EM side-channel leakages; thus, classic methods like shuffling and masking could be employed to mitigate these attacks by reducing oracle accuracy. However, the security and concrete performance of protected implementations warrant further investigation.

Developing secure and efficient HQC implementations for the ARM Cortex-M4 platform is particularly challenging. The newly discovered long and strong leakage profile for the FD attack on HQC exacerbates this difficulty. Future research should prioritize the development and evaluation of masked implementations to assess their computational overhead and effectiveness in resisting our new FD attacks.

Finally, exploring detection-based countermeasures [RCDB24] that identify and reject malicious ciphertexts remains an active research area, although challenges in effectiveness and avoiding the introduction of new vulnerabilities exist.

5.4 Comparison with Recent Works

Table 7 compares our attacks with other relevant side-channel attacks against HQC. The first three attacks (OT-PCA [DG25], MV-PC, and FD) are oracle-based, listed in decreasing order of generality and increasing efficiency in oracle calls. While OT-PCA [DG25] offers broader applicability due to the more general nature and ease of construction of PCOs (e.g., in microarchitecture settings [SGG24], where MV-PC and FD oracle attacks are currently unexplored), our new MV-PC and, especially, FD attacks demonstrate significantly higher efficiency. The MV-PC attack is more general than the FD attack as it has implications for parallel key mismatch attacks and fault-injection attacks, unlike the FD attack.

Direct comparisons of trace counts can be misleading due to variations in exploited leakages and attack scenarios. For example, Paiva et al. [PRJB23] present a two-trace key-recovery attack on unprotected Cortex-M4 platforms. Their method utilizes only valid ciphertexts, effectively bypassing security measures that rely on detecting decryption failures. However, this attack necessitates recovering extensive information—on the order of ten thousand bits—corresponding to the vector $x \cdot r_2 + r_1 \cdot y$ from relatively short leakage intervals [PRJB23].

In contrast, our FD oracle-based methodology requires substantially less information—approximately 2.8 bits per Reed-Muller output for hqc-128 (as discussed previously), totaling around 260 bits across the two required traces—but can successfully launch the attack by exploiting leakage traces corresponding to as few as three out of the thirty iterations within the `compute_syndromes` function to obtain this required information. The primary bottleneck thus differs between the approaches: noise significantly impacts the feasibility of a single-trace attack for Paiva et al. [PRJB23], whereas our technique is fundamentally limited by the oracle interaction needed to acquire information for x and y separately. From this perspective, we argue that our attack framework may prove more potent against future, well-masked implementations where side-channel leakage, although significantly

attenuated, might still be present.

Furthermore, our FD attacks are intrinsically oracle-based, with the oracle construction rooted in the separate processing of n_1 Reed-Muller decoding outputs. This foundation suggests that our FD attacks could possess broader applicability across various platforms and could potentially be constructed using different leakage sources, beyond the two currently identified (one at the beginning stage of Reed-Solomon decoding and another in Reed-Muller decoding).

Goy et al. [GMGL24] achieve single-trace message recovery using Soft Analytic Side-Channel Attacks (SASCA) [VGS14], targeting a weaker objective than key recovery. Their SASCA approach offers an alternative to our deep-learning-based oracle construction, presenting a promising avenue for future research.

Goy et al. [GMGL24] also demonstrate the vulnerability of codeword masking [MSS13] to their proposed SASCA techniques when applied to HQC. This result is particularly relevant as it suggests that similar analytical side-channel methods could potentially be adapted to construct the FD oracles required by our attack framework. Consequently, the applicability of our FD attacks may extend beyond the unmasked implementations evaluated herein, potentially targeting implementations protected by codeword masking, provided such oracles can be realized. This further highlights the generic, oracle-based nature of our proposed FD attacks, whose feasibility hinges on the existence of a suitable oracle rather than being intrinsically limited to specific leakage characteristics.

6 Concluding Remarks

This paper has pioneered the first MV-PC and FD oracle attacks on HQC, a significant advancement in code-based cryptography. Our MV-PC attack, extending the OT-PCA framework [DG25], requires only 60% of oracle queries for hqc-128 compared to state-of-the-art PC oracle attacks. The new MV-PC attack holds significance for side-channel, parallel key mismatch, and fault-injection analyses. The novel FD attack exhibits exceptional efficiency, recovering the hqc-128 secret key with only two queries to a perfect oracle (four for hqc-192/hqc-256). Importantly, both attacks are robust against imperfect oracle responses. Experimental validation on an ARM Cortex-M4 microcontroller confirms the feasibility of constructing accurate MV-PC and FD oracles from power leakage.

Given the demonstrated vulnerabilities, and particularly since HQC has been selected for standardization by NIST, securing HQC implementations is paramount. Future research should prioritize the development and rigorous evaluation of higher-order masking techniques specifically designed for HQC's unique structure, addressing the substantial leakage observed during Reed-Solomon decoding's syndrome computation, as demonstrated by our FD attack. The significant performance overhead of high-order masking on resource-constrained platforms like the ARM Cortex-M4 calls for thorough investigation. Exploring alternative countermeasures, such as detection-based mechanisms [RCDB24], and extending this research to other code-based KEMs are crucial next steps. A comprehensive comparative analysis of various countermeasure approaches and their resilience against both MV-PC and FD attacks is needed to guide the development of secure and efficient HQC implementations. Last, the oracle-based nature of our attacks encourages further research into identifying new leakage sources. In well-protected implementations, such as those with masking, a single leakage source may only provide minor information, making it insufficient for creating a precise oracle. Future work should therefore investigate methods for combining multiple leakage sources to construct more accurate and effective oracles.

Acknowledgements

This work was supported by the Swedish Research Council (grant numbers 2021-04602 and 2023-03654), the Swedish Civil Contingencies Agency (grant number 2020-11632), the Crafoord Foundation, and the Wallenberg AI, Autonomous Systems and Software Program (WASP), funded by the Knut and Alice Wallenberg Foundation. The computations and simulations were partly enabled by resources provided by LUNARC.

References

- [AAB⁺19] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, and Gilles Zémor. HQC. Technical report, National Institute of Standards and Technology, 2019. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-2-submissions>.
- [AAB⁺22] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, and Pascal Veron. HQC. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [ABB⁺22] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Phillippe Gaborit, Shay Gueron, Tim Guneyasu, Carlos Aguilar-Melchor, Rafael Misoczki, Edoardo Persichetti, Nicolas Sendrier, Jean-Pierre Tillich, Gilles Zémor, Valentin Vasseur, Santosh Ghosh, and Jan Richter-Brokmann. BIKE. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [ABC⁺22] Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
- [ABC⁺25] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the fourth round of the NIST post-quantum cryptography standardization process. *National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [ABP15] Michel Abdalla, Fabrice Benhamouda, and David Pointcheval. Public-key encryption indistinguishable under plaintext-checkable attacks. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 332–352. Springer, Berlin, Heidelberg, March / April 2015.
- [AD97] Miklós Ajtai and Cynthia Dwork. A public-key cryptosystem with worst-case/average-case equivalence. In *29th ACM STOC*, pages 284–293. ACM Press, May 1997.

- [AG19] Nicolas Aragon and Philippe Gaborit. A key recovery attack against LRPC using decryption failures. In *International Workshop on Coding and Cryptography (WCC)*, Saint-Jacut-de-la-Mer, France, 2019.
- [BBB⁺24] Daniel J. Bernstein, Karthikeyan Bhargavan, Shivam Bhasin, Anupam Chattopadhyay, Tee Kiah Chia, Matthias J. Kannwischer, Franziskus Kiefer, Thales Paiva, Prasanna Ravi, and Goutam Tamvada. KyberSlash: Exploiting secret-dependent division timings in kyber implementations. Cryptology ePrint Archive, Report 2024/1049, 2024.
- [BDH⁺19] Ciprian Băetu, F. Betül Durak, Loïs Huguenin-Dumittan, Abdullah Talayhan, and Serge Vaudenay. Misuse attacks on post-quantum cryptosystems. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 747–776. Springer, Cham, May 2019.
- [BGRR19] Aurélie Bauer, Henri Gilbert, Guénaél Renault, and Mélissa Rossi. Assessment of the key-reuse resilience of NewHope. In Mitsuru Matsui, editor, *CT-RSA 2019*, volume 11405 of *LNCS*, pages 272–292. Springer, Cham, March 2019.
- [Bid22] Joseph Biden. National security memorandum on promoting united states leadership in quantum computing while mitigating risks to vulnerable cryptographic systems, May 4 2022.
- [CCCM24] Bhuvnesh Chaturvedi, Anirban Chakraborty, Ayantika Chatterjee, and Debdeep Mukhopadhyay. “Ask and thou shall receive”: Reaction-based full key recovery attacks on FHE. In Joaquin Garcia-Alfaro, Rafał Kozik, Michał Choraś, and Sokratis Katsikas, editors, *ESORICS 2024, Part IV*, volume 14985 of *LNCS*, pages 457–477. Springer, Cham, September 2024.
- [CWS⁺24] Boru Chen, Yingchen Wang, Pradyumna Shome, Christopher W. Fletcher, David Kohlbrenner, Riccardo Paccagnella, and Daniel Genkin. Gofetch: Breaking constant-time cryptographic implementations using data memory-dependent prefetchers. In *USENIX Security*, 2024.
- [DFR18] Jintai Ding, Scott R. Fluhrer, and Saraswathy RV. Complete attack on RLWE key exchange with reused keys, without signal leakage. In Willy Susilo and Guomin Yang, editors, *ACISP 18*, volume 10946 of *LNCS*, pages 467–486. Springer, Cham, July 2018.
- [DG25] Haiyue Dong and Qian Guo. OT-PCA: New key-recovery plaintext-checking oracle based side-channel attacks on HQC with offline templates. In *IACR TCHES*, 2025.
- [DGJ⁺19] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption failure attacks on IND-CCA secure lattice-based schemes. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part II*, volume 11443 of *LNCS*, pages 565–598. Springer, Cham, April 2019.
- [DNG22] Elena Dubrova, Kalle Ngo, and Joel Gärtner. Breaking a fifth-order masked implementation of CRYSTALS-Kyber by copy-paste. Cryptology ePrint Archive, Report 2022/1713, 2022.
- [DRV20] Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 3–33. Springer, Cham, May 2020.

- [DTVV19] Jan-Pieter D’Anvers, Marcel Tiepelt, Frederik Vercauteren, and Ingrid Verbauwhede. Timing attacks on error correcting codes in post-quantum schemes. In *Proceedings of ACM Workshop on Theory of Implementation Security Workshop*, pages 2–9, 2019.
- [ELPS18] Edward Eaton, Matthieu Lequesne, Alex Parent, and Nicolas Sendrier. QC-MDPC: A timing attack and a CCA2 KEM. In Tanja Lange and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018*, pages 47–76. Springer, Cham, 2018.
- [FHS⁺17] Tomás Fabsic, Viliam Hromada, Paul Stankovski, Pavol Zajac, Qian Guo, and Thomas Johansson. A reaction attack on the QC-LDPC McEliece cryptosystem. In Tanja Lange and Tsuyoshi Takagi, editors, *Post-Quantum Cryptography - 8th International Workshop, PQCrypto 2017*, pages 51–68. Springer, Cham, 2017.
- [Flu16] Scott Fluhrer. Cryptanalysis of ring-LWE based key exchange with key share reuse. Cryptology ePrint Archive, Report 2016/085, 2016.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999.
- [FS09] Matthieu Finiasz and Nicolas Sendrier. Security bounds for the design of code-based cryptosystems. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 88–105. Springer, Berlin, Heidelberg, December 2009.
- [GHJ⁺22] Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder. Don’t reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE. *IACR TCHES*, 2022(3):223–263, 2022.
- [GJ20] Qian Guo and Thomas Johansson. A new decryption failure attack against HQC. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part I*, volume 12491 of *LNCS*, pages 353–382. Springer, Cham, December 2020.
- [GJN19] Qian Guo, Thomas Johansson, and Alexander Nilsson. A generic attack on lattice-based schemes using decryption errors with application to ss-ntru-pke. Cryptology ePrint Archive, Report 2019/043, 2019.
- [GJN20] Qian Guo, Thomas Johansson, and Alexander Nilsson. A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 359–386. Springer, Cham, August 2020.
- [GJS16] Qian Guo, Thomas Johansson, and Paul Stankovski. A key recovery attack on MDPC with CCA security using decoding errors. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 789–815. Springer, Berlin, Heidelberg, December 2016.
- [GJY19] Qian Guo, Thomas Johansson, and Jing Yang. A novel CCA attack using decryption errors against LAC. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part I*, volume 11921 of *LNCS*, pages 82–111. Springer, Cham, December 2019.

- [GLG22] Guillaume Goy, Antoine Loiseau, and Philippe Gaborit. A new key recovery side-channel attack on HQC with chosen ciphertext. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings*, volume 13512 of *Lecture Notes in Computer Science*, pages 353–371. Springer, 2022.
- [GM23] Qian Guo and Erik Mårtensson. Do not bound to a single position: Near-optimal multi-positional mismatch attacks against kyber and saber. In Thomas Johansson and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023*, pages 291–320. Springer, Cham, August 2023.
- [GMÅ24] Qian Guo, Erik Mårtensson, and Adrian Åström. The perils of limited key reuse: Adaptive and parallel mismatch attacks with post-processing against kyber. *CiC*, 1(3):21, 2024.
- [GMGL24] Guillaume Goy, Julien Maillard, Philippe Gaborit, and Antoine Loiseau. Single trace HQC shared key recovery with SASCA. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(2):64–87, 2024.
- [GMR20] Aurélien Greuet, Simon Montoya, and Guénaél Renault. Attack on LAC key exchange in misuse situation. In Stephan Krenn, Haya Shulman, and Serge Vaudenay, editors, *CANS 20*, volume 12579 of *LNCS*, pages 549–569. Springer, Cham, December 2020.
- [GN07] Nicolas Gama and Phong Q. Nguyen. New chosen-ciphertext attacks on NTRU. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007*, volume 4450 of *LNCS*, pages 89–106. Springer, Berlin, Heidelberg, April 2007.
- [GNNJ23] Qian Guo, Denis Nabokov, Alexander Nilsson, and Thomas Johansson. SCA-LDPC: A code-based framework for key-recovery side-channel attacks on post-quantum encryption schemes. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part IV*, volume 14441 of *LNCS*, pages 203–236. Springer, Singapore, December 2023.
- [GWSG25] Stefan Gast, Hannes Weissteiner, Robin Leander Schröder, and Daniel Gruss. CounterSEVeillance: Performance-counter attacks on AMD SEV-SNP. In *Network and Distributed System Security Symposium 2025: NDSS 2025*, 2025.
- [HGS99] Chris Hall, Ian Goldberg, and Bruce Schneier. Reaction attacks against several public-key cryptosystems. In Vijay Varadharajan and Yi Mu, editors, *ICICS 99*, volume 1726 of *LNCS*, pages 2–12. Springer, Berlin, Heidelberg, November 1999.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017.
- [HNP⁺03] Nick Howgrave-Graham, Phong Q. Nguyen, David Pointcheval, John Proos, Joseph H. Silverman, Ari Singer, and William Whyte. The impact of decryption failures on the security of NTRU encryption. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 226–246. Springer, Berlin, Heidelberg, August 2003.

- [HSC⁺23] Senyang Huang, Rui Qi Sim, Chitchanok Chuengsatiansup, Qian Guo, and Thomas Johansson. Cache-timing attack against HQC. *IACR TCHES*, 2023(3):136–163, 2023.
- [JAC⁺22] David Jao, Reza Azarderakhsh, Matthew Campagna, Craig Costello, Luca De Feo, Basil Hess, Amir Jalali, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Joost Renes, Vladimir Soukharev, David Urbanik, Geovandro Pereira, Koray Karabina, and Aaron Hutchinson. SIKE. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [JJ00] Éliane Jaulmes and Antoine Joux. A chosen-ciphertext attack against NTRU. In Mihir Bellare, editor, *CRYPTO 2000*, volume 1880 of *LNCS*, pages 20–35. Springer, Berlin, Heidelberg, August 2000.
- [KSSW22] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving software quality in cryptography standardization projects. In *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, pages 19–30, Los Alamitos, CA, USA, 2022. IEEE Computer Society.
- [McE78] Robert J. McEliece. A public key cryptosystem based on algebraic coding theory. *DSN Progress Report*, pages 114–116, 1978.
- [MKB⁺24] Puja Mondal, Suparna Kundu, Sarani Bhattacharya, Angshuman Karmakar, and Ingrid Verbauwhede. A practical key-recovery attack on LWE-based key-encapsulation mechanism schemes using rowhammer. In Christina Pöpper and Lejla Batina, editors, *ACNS 24 International Conference on Applied Cryptography and Network Security, Part III*, volume 14585 of *LNCS*, pages 271–300. Springer, Cham, March 2024.
- [MSS13] Dominik Merli, Frederic Stumpf, and Georg Sigl. Protecting PUF error correction by codeword masking. *Cryptology ePrint Archive*, Paper 2013/334, 2013.
- [Nat24] National Institute of Standards and Technology. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. Technical report, Federal Information Processing Standards Publication, Gaithersburg, MD 20899-8900, August 2024. Category: Computer Security, Subcategory: Cryptography.
- [NDGJ21] Kalle Ngo, Elena Dubrova, Qian Guo, and Thomas Johansson. A side-channel attack on a masked IND-CCA secure Saber KEM implementation. *IACR TCHES*, 2021(4):676–707, 2021.
- [NDJ23] Kalle Ngo, Elena Dubrova, and Thomas Johansson. A side-channel attack on a masked and shuffled software implementation of Saber. *Journal of Cryptographic Engineering*, 13(4):443–460, November 2023.
- [NJW18] Alexander Nilsson, Thomas Johansson, and Paul Stankovski Wagner. Error amplification in code-based cryptography. *IACR TCHES*, 2019(1):238–258, 2018.
- [OP01] Tatsuaki Okamoto and David Pointcheval. REACT: Rapid Enhanced-security Asymmetric Cryptosystem Transform. In David Naccache, editor, *CT-RSA 2001*, volume 2020 of *LNCS*, pages 159–175. Springer, Berlin, Heidelberg, April 2001.

- [OWT20] Satoshi Okada, Yuntao Wang, and Tsuyoshi Takagi. Improving key mismatch attack on NewHope with fewer queries. In Joseph K. Liu and Hui Cui, editors, *ACISP 20*, volume 12248 of *LNCS*, pages 505–524. Springer, Cham, November / December 2020.
- [PRJB23] Thales Paiva, Prasanna Ravi, Dirmanto Jap, and Shivam Bhasin. Et tu, brute? SCA assisted CCA using valid ciphertexts - A case study on HQC KEM. Cryptology ePrint Archive, Report 2023/1626, 2023.
- [PT19] Thales Bandiera Paiva and Routo Terada. A timing attack on the HQC encryption scheme. In Kenneth G. Paterson and Douglas Stebila, editors, *SAC 2019*, volume 11959 of *LNCS*, pages 551–573. Springer, Cham, August 2019.
- [QCD19] Yue Qin, Chi Cheng, and Jintai Ding. A complete and optimized key mismatch attack on NIST candidate NewHope. In Kazue Sako, Steve Schneider, and Peter Y. A. Ryan, editors, *ESORICS 2019, Part II*, volume 11736 of *LNCS*, pages 504–520. Springer, Cham, September 2019.
- [QCZ⁺21] Yue Qin, Chi Cheng, Xiaohan Zhang, Yanbin Pan, Lei Hu, and Jintai Ding. A systematic approach and analysis of key mismatch attacks on lattice-based NIST candidate KEMs. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 92–121. Springer, Cham, December 2021.
- [RCDB24] Prasanna Ravi, Anupam Chattopadhyay, Jan Pieter D’Anvers, and Anubhab Bakshi. Side-channel and fault-injection attacks over lattice-based post-quantum schemes (Kyber, Dilithium): Survey and new results. *ACM Transactions on Embedded Computing Systems*, 23(2):1–54, 2024.
- [REB⁺22] Prasanna Ravi, Martianus Frederic Ezerman, Shivam Bhasin, Anupam Chattopadhyay, and Sujoy Sinha Roy. Will you cross the threshold for me? Generic side-channel assisted chosen-ciphertext attacks on NTRU-based KEMs. *IACR TCHES*, 2022(1):722–761, 2022.
- [RRCB20] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR TCHES*, 2020(3):307–335, 2020.
- [RRD⁺23] Gokulnath Rajendran, Prasanna Ravi, Jan-Pieter D’Anvers, Shivam Bhasin, and Anupam Chattopadhyay. Pushing the limits of generic side-channel attacks on LWE-based KEMs - parallel PC oracle attacks on Kyber KEM and beyond. *IACR TCHES*, 2023(2):418–446, 2023.
- [SAB⁺22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [SGG24] Robin Leander Schröder, Stefan Gast, and Qian Guo. Divide and surrender: Exploiting variable division instruction timing in HQC key recovery attacks. In *USENIX Security*. USENIX Association, 2024.

- [SHR⁺22] Thomas Schamberger, Lukas Holzbaur, Julian Renner, Antonia Wachter-Zeh, and Georg Sigl. A Power Side-Channel Attack on the Reed-Muller Reed-Solomon Version of the HQC Cryptosystem. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings*, volume 13512 of *Lecture Notes in Computer Science*, pages 327–352. Springer, 2022.
- [SLZ24] Mingyao Shao, Yuejun Liu, and Yongbin Zhou. Pairwise and parallel: Enhancing the key mismatch attacks on kyber and beyond. In Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. Cárdenas, editors, *ASIACCS 24*. ACM Press, July 2024.
- [TUX⁺23] Yutaro Tanaka, Rei Ueno, Keita Xagawa, Akira Ito, Junko Takahashi, and Naofumi Homma. Multiple-valued plaintext-checking side-channel attacks on post-quantum KEMs. *IACR TCHES*, 2023(3):473–503, 2023.
- [UXT⁺22] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. Curse of re-encryption: A generic power/EM analysis on post-quantum KEMs. *IACR TCHES*, 2022(1):296–322, 2022.
- [VGS14] Nicolas Veyrat-Charvillon, Benoît Gérard, and François-Xavier Standaert. Soft analytical side-channel attacks. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part I*, volume 8873 of *LNCS*, pages 282–296. Springer, Berlin, Heidelberg, December 2014.
- [WPH⁺22] Yingchen Wang, Riccardo Paccagnella, Elizabeth Tang He, Hovav Shacham, Christopher W. Fletcher, and David Kohlbrenner. Hertzbleed: Turning power side-channel attacks into remote timing attacks on x86. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 679–697. USENIX Association, August 2022.
- [WTBB⁺19] Guillaume Wafo-Tapa, Slim Bettaiieb, Loic Bidoux, Philippe Gaborit, and Etienne Marcatel. A practicable timing attack against HQC and its counter-measure. Cryptology ePrint Archive, Report 2019/909, 2019.
- [WWW23] Tianrui Wang, Anyu Wang, and Xiaoyun Wang. Exploring decryption failures of BIKE: New class of weak keys and key recovery attacks. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 70–100. Springer, Cham, August 2023.
- [XIU⁺21] Keita Xagawa, Akira Ito, Rei Ueno, Junko Takahashi, and Naofumi Homma. Fault-injection attacks against NIST’s post-quantum cryptography round 3 KEM candidates. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 33–61. Springer, Cham, December 2021.
- [XPRO20] Zhuang Xu, Owen Pemberton, Sujoy Sinha Roy, and David F. Oswald. Magnifying Side-Channel Leakage of Lattice-Based Cryptosystems with Chosen Ciphertexts: The Case Study of Kyber. *IACR Cryptol. ePrint Arch.*, page 912, 2020.
- [ZPS12] Zhenfei Zhang, Thomas Plantard, and Willy Susilo. Reaction attack on outsourced computing with fully homomorphic encryption schemes. In Howon Kim, editor, *ICISC 11*, volume 7259 of *LNCS*, pages 419–436. Springer, Berlin, Heidelberg, November / December 2012.

A Simulation Results for hqc-192 and hqc-256

The simulated mean numbers of errors and success rates for hqc-192 and hqc-256 for the new MV-PC and FD methods are reported in Table 8.

To calculate the success rates, the post-processing parameters are set in the following way: $\ell = 61$, $T_0 = 24000$, $T_1 = 18000$, and $n_{\text{iter}} = 16$ for hqc-192, and $\ell = 61$, $T_0 = 40000$, $T_1 = 30000$, and $n_{\text{iter}} = 16$ for hqc-256. The post-processing complexity for hqc-192 is only slightly higher than in [DG25] as the parameter T_1 is decreased by 2000 to 18 000. For example, the main complexity term, defined in Equation (6), is unchanged when rounded to the nearest integer power of 2. The post-processing parameters and complexity for hqc-256 are the same as in [DG25].

Table 8: Simulation results for hqc-192 and hqc-256 with perfect oracle accuracy ($\rho = 1$). Panel (a) presents results for the MV-PC method, where the number of oracle calls required is $2t \cdot n_1$. Panel (b) shows results for the FD method, where the number of oracle calls is $2t$. The parameter t represents the number of error patterns. The “Mean Number of Errors” is the average count of errors in the initial n positions of the reordered z vector. Success rates (P_{success}) are provided for two cases: $w_0 = 2$ and $w_0 = 3$, where w_0 is defined in Section 2.3. Each scenario is evaluated using 10 000 randomly generated keys.

(a) MV-PC: The number of MV-PC oracle calls is $2t \cdot n_1$.

	t	Mean Number of Errors	Success Rate (P_{success})	
			$w_0 = 2$	$w_0 = 3$
hqc-192	5	3.2866	64.22%	77.79%
hqc-256	4	3.4414	58.95%	73.34%

(b) FD: The number of FD oracle calls is $2t$.

	t	Mean Number of Errors	Success Rate (P_{success})	
			$w_0 = 2$	$w_0 = 3$
hqc-192	2	4.0046	47.12%	64.59%
hqc-256	2	3.2656	60.26%	72.71%

B Supplementary Details for MV-PC Attack Validation

Leakage analysis is conducted on power traces collected during the execution of the PQCLEAN_HQC128_CLEAN_code_encode function as part of the MV-PC attack validation in Section 4.2 of the main text. The collected traces exhibit variations depending on the decrypted message m' being processed. We first collect traces corresponding to the processing of each m' and identify strong leakage points. We analyze 130 000 consecutive data points starting from the 751 000th timestamp of all traces. Pairwise Test Vector Leakage Assessment (TVLA) is conducted to identify leakage points during computation. Figure 6a illustrates the mean trace difference for two distinct m' across the first 6 000 sample points, while Figure 6b illustrates TVLA results for the entire trace. In total, all pairwise TVLAs yield 5 607 strong leakage point positions (t -test values larger than 10) per trace.

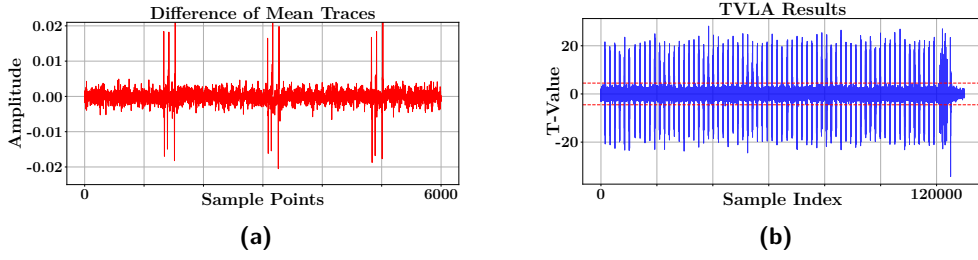


Figure 6: Leakage analysis based on power traces of the encoding function in the re-encryption phase of KEM.Decaps. Figures and tests are based on data from 100 power traces per m' for two distinct decrypted messages m' . Only partial traces starting from the 751 000th clock cycle are used for the plots. Figure 6a includes samples of the first 6 000 consecutive data points, while Figure 6b includes a total of 130 000 consecutive data points.

C Supplementary Details for FD Attack Validation

In this section, we provide additional details on the leakage detection and analysis for the FD attack validation, as described in Section 4.3 of the main text. Specifically, we begin with the leakage analysis for the Reed-Solomon decoding procedure and explain how the trace slices corresponding to the processing of each block in the Reed-Solomon decoding procedure are identified, which is necessary for training a separate NN model for each block. Next, we describe the process of identifying leakage in the inner Reed-Muller decoding procedure, followed by its leakage analysis.

C.1 Leakage Analysis for Reed-Solomon Decoding Procedure

We start with constructing the desired codewords to collect traces for TVLA. As our selected error pattern has 27 types of outputs from the Reed-Muller decoder, we sample key blocks ξ , each of length n_2 and according to the secret key distribution, and collect 100 ξ for each output type. Each ξ is then XORed with the optimized error pattern, and the resulting vector is padded into a codeword with length $n_1 \cdot n_2$, called $\text{cdw}_{\text{padded}}$. In other words, each of the n_1 blocks constituting the codeword $\text{cdw}_{\text{padded}}$ contains the same vector of length- n_2 and will be decoded to the same output by the Reed-Muller decoder. We collect traces from the Reed-Solomon decoding phase for these codewords and conduct pairwise TVLA.

Figure 7a shows the first 6 000 consecutive data points of the difference between the mean traces for two distinct Reed-Muller decoding outputs, starting from the 910 050th timestamp. Figure 7b shows the TVLA results between traces for these two messages based on 130 000 consecutive data points starting from the 910 050th timestamp.

To identify the trace slice corresponding to the computation of each block, we construct the codeword in the following way. For certain Reed-Muller decoding output type, each length- n_2 vector, obtained from XORing the sampled ξ with the error pattern, is positioned into the last block of a codeword with n_1 blocks. The first $n_1 - 1$ blocks of this codeword are set to all zeros. We collect traces from the Reed-Solomon decoding phase for this type of codewords. We compare the traces for this type of codewords with those for $\text{cdw}_{\text{padded}}$ constructed in the previous subsection for the same Reed-Muller decoding output type using TVLA. Note that for these two types of codewords, only the last block is the same and will be decoded to the same output by the Reed-Muller decoder. We expect that the t -test values for traces for the last block to be very low, while that for traces for the other blocks to be high. Figure 7c shows the TVLA result between traces for these two types of

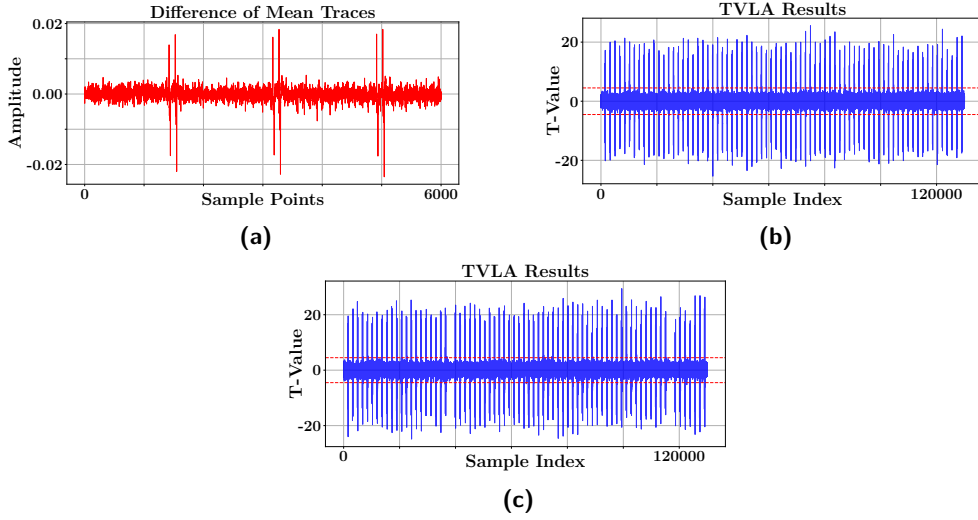


Figure 7: Leakage analysis for the Reed-Solomon decoding function during KEM.Decaps. Figures and tests are based on data from 100 power traces per output for two distinct Reed-Muller decoding outputs. Only partial traces are shown, starting from the 910 050th clock cycle. Figure 7a includes samples of the first 6 000 consecutive data points, while Figures 7b and 7c include a total of 130 000 consecutive data points.

codewords. Indeed, we are able to observe two gaps in the t -test value plot when compared with the previous TVLA result in Figure 7b. The first gap marks a rough index range in the trace corresponding to the computation for the last block in the second iteration of the Reed-Solomon decoding phase, and the second gap marks a rough index range for the last block in the third iteration.

For each block in each computation iteration, we determine a range of position indices to take traces from by combining the strongest leakage points (t -test values larger than 7.5) from all pairwise TVLA and the rough index range of each block. For each block, we then collect three trace slices, each covering the strongest leakage points from one of the three iterations. For the first 23 blocks, the number of total data points is 886. For the rest 22 blocks, it is 902.

C.2 Leakage Detection and Analysis for Reed-Muller Decoding Procedure

The inner Reed-Muller decoding procedure, which precedes Reed-Solomon decoding, constitutes another leakage source for constructing an FD oracle. Listing 2 presents its implementation from the PQClean library. This procedure iterates n_1 times, with each iteration corresponding to the decoding of a single Reed-Muller codeword. The desired values for building the FD oracle are the outputs $\text{msg}[i]$ for $i \in [0..n_1]$. We find that while this procedure leaks all n_1 outputs, the leakage is weaker than that observed in the Reed-Solomon decoding stage. Specifically, the leakage originates from the `find_peaks` function, which computes the location of the highest value after the previous operations. This location directly corresponds to the output of the Reed-Muller decoder, thereby providing the information needed to construct the oracle.

```

1 void PQCLEAN_HQC128_CLEAN_reed_muller_decode(uint8_t *msg, const uint64_t
   *cdw) {
2     uint16_t expanded[128];
3     uint16_t transform[128];

```

```

4   for (size_t i = 0; i < VEC_N1_SIZE_BYTES; ++i) {
5       expand_and_sum(expanded, &cdw[2 * i * MULTIPLICITY]);
6       hadamard(expanded, transform);
7       transform[0] -= 64 * MULTIPLICITY;
8       msg[i] = find_peaks(transform);
9   }
10 }

```

Listing 2: Reed-Muller decoding.

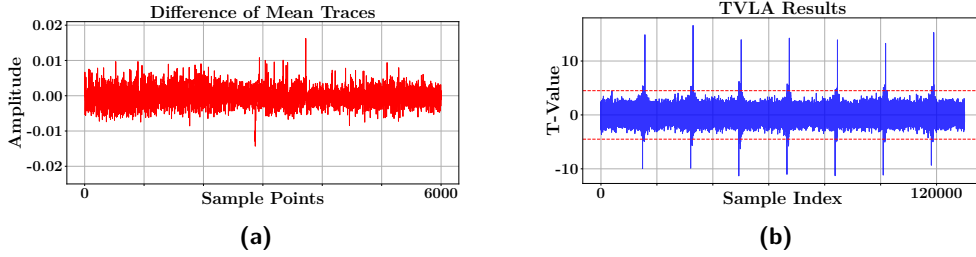


Figure 8: Leakage analysis for the Reed-Muller decoding function during KEM.Decaps. Figures and tests are based on data from 100 power traces per output for two distinct decoding outputs. Only partial traces are shown, starting from the first clock cycle. Figure 8a includes samples of 6 000 consecutive data points between indices 12 000 and 18 000, while Figure 8b includes a total of 130 000 consecutive data points starting from the first clock cycle.

To identify the strongest leakage points, we use the same codewords as in the Reed-Solomon leakage detection and collect 130 000 consecutive data points from the initial timestamp of each trace. Pairwise TVLA is then conducted on traces corresponding to different Reed-Muller decoding outputs. Figure 8a illustrates the difference in means of 6 000 data points (indices 12 000 to 18 000) between traces of two distinct decoding outputs. Figure 8b presents the TVLA results for these two traces. The figures demonstrate that information leaked from the Reed-Muller decoding procedure is relatively weak and sparse.

Within the 130 000 consecutive data points, the computations for the first seven Reed-Muller blocks are present. As shown in Listing 2, each iteration within the Reed-Muller procedure involves identical computations. Cross-correlation analysis identifies this repetition in the traces, revealing that each block corresponds to 17 210 points. Although only information from the first block is needed, traces encompassing all seven blocks are collected to obtain additional samples for training the deep learning model for the first block. Ultimately, 3 468 data points with TVLA values exceeding 4.5 are selected.